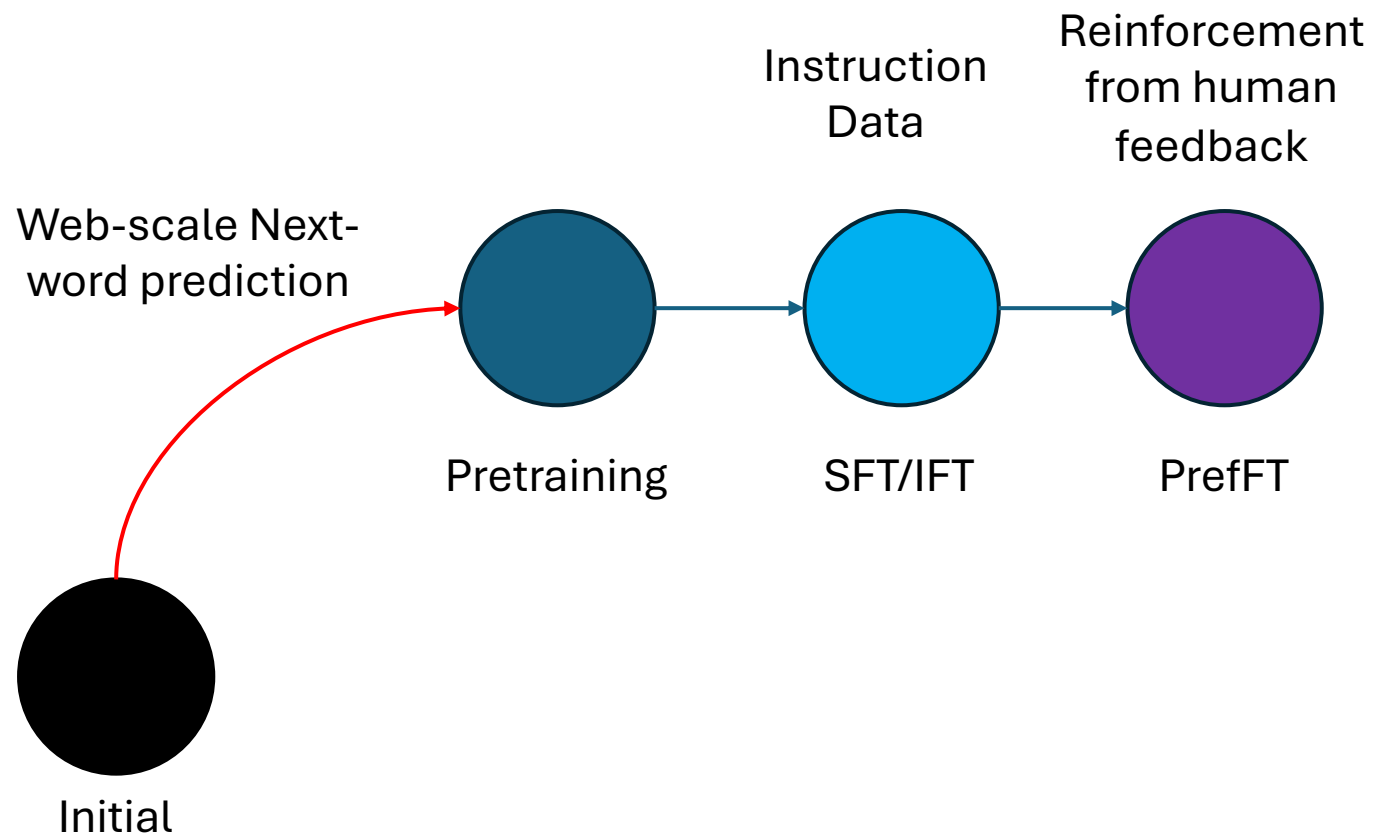


Deep Learning

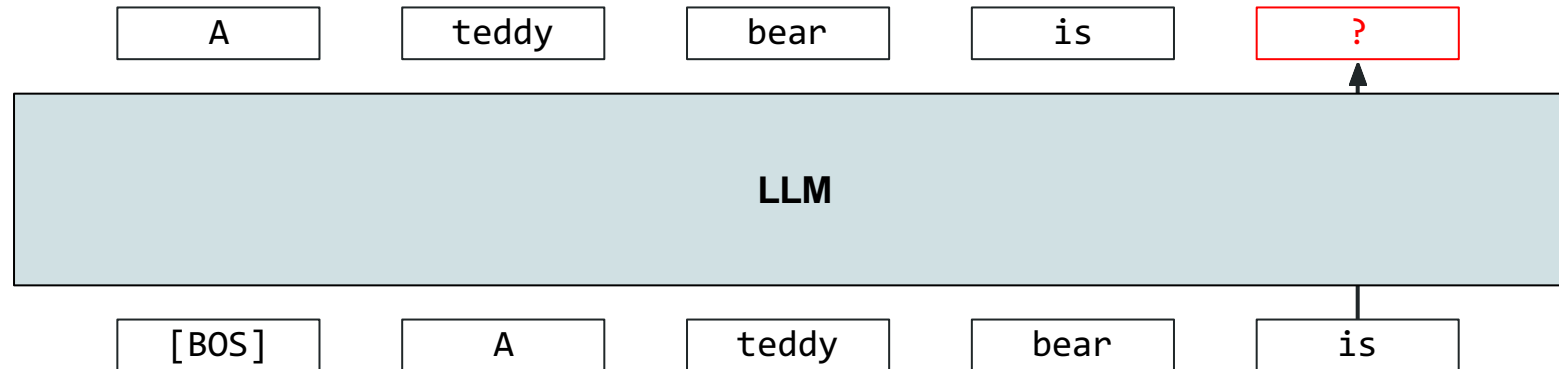
Lecture 11

The path to ChatGPT



Modelling “language” probability

- Objective function. Predict next token

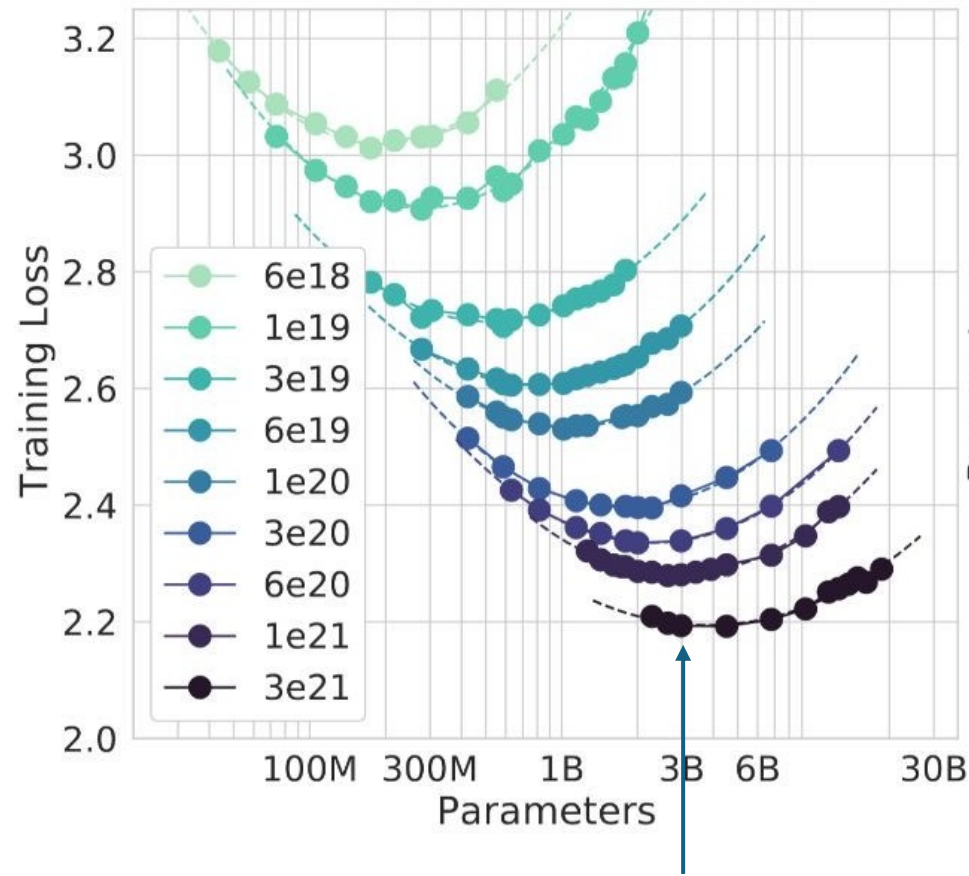


Pretraining scale

- **Size.** Approx. trillions of tokens

Model	Pretraining size (# tokens)
GPT-3	300 billion
LLaMA 3	15 trillion

Chinchilla law



We observe the trend that more data leads to lower loss, but without increasing the data, there is a limit

Given #tokens, each model has an optimal number of parameters

Ideal points

Parameters	FLOPs	FLOPs (in <i>Gopher</i> unit)	Tokens
400 Million	1.92e+19	1/29,968	8.0 Billion
1 Billion	1.21e+20	1/4,761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion

Scaling laws: predicting loss from compute, data and parameters

- Empirical **power laws**: test loss falls as a straight line in log-log with data, parameters, and compute (Kaplan et al., 2020)
- Chinchilla (Hoffmann et al., 2022): for a fixed compute budget there is an **optimal model size and token count** (≈ 20 tokens/parameter)
- Why we care: decide *bigger model vs. more data*, pick hyperparameters on small models and transfer up, and forecast what a training run will give
- The next slides follow Stanford CS336 (Lecture 9 and 11, Spring 2025)

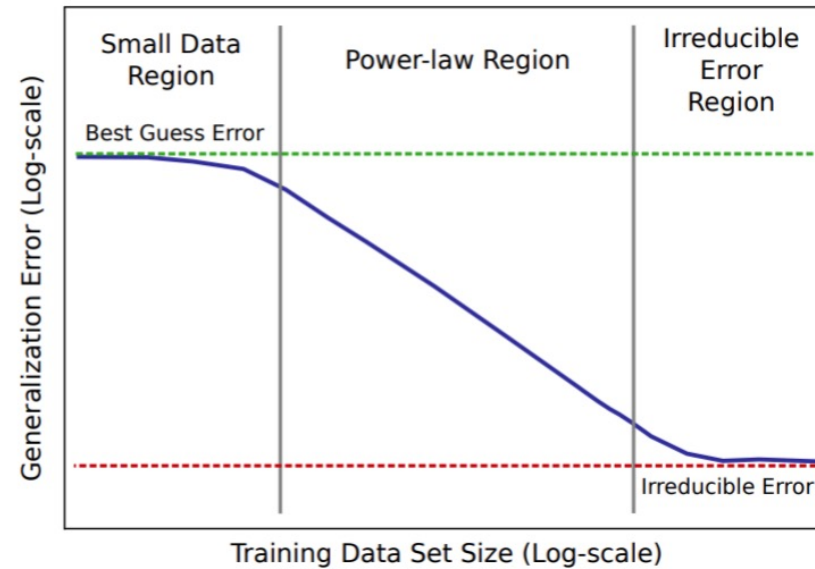
Data vs performance

What's a data scaling law?

Data scaling laws : simple formula that maps dataset size (n) to error

What do we expect out of scaling laws?

Monotonic, logistic-like curves

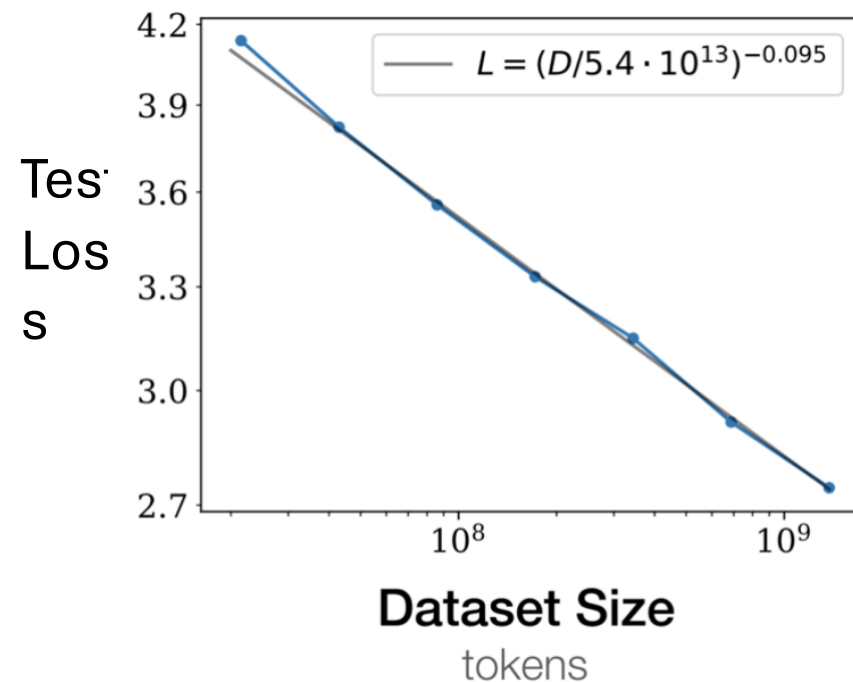


[Hestness+ 2017]

Data scaling laws for language models

First, an empirical observation

Loss and dataset size is linear on a log-log plot

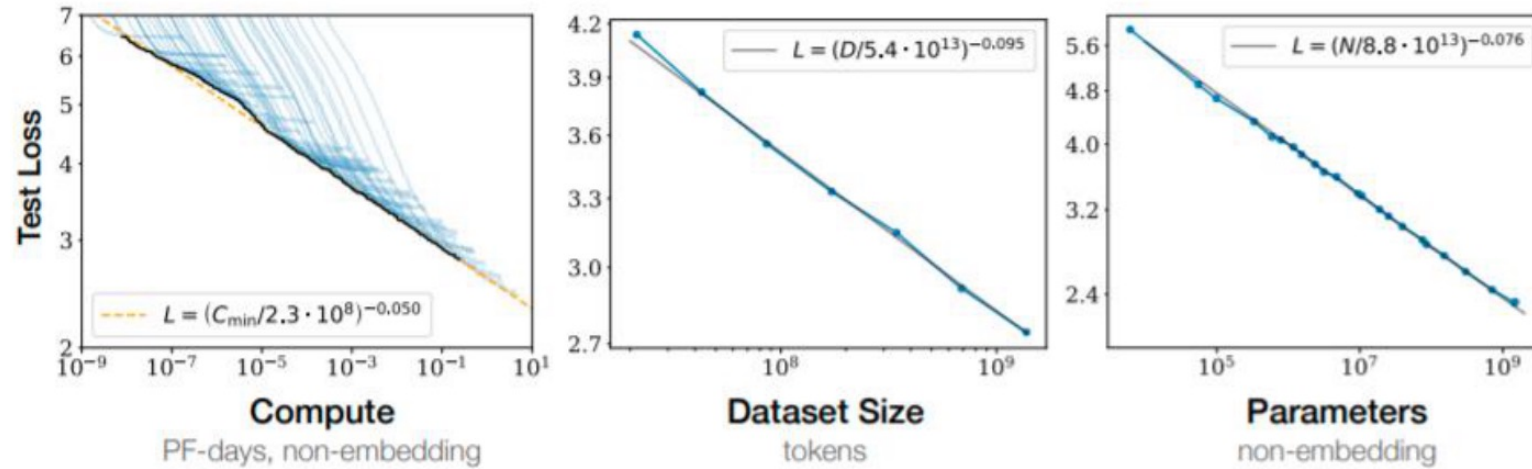


“Scale-free” or
“Power law”

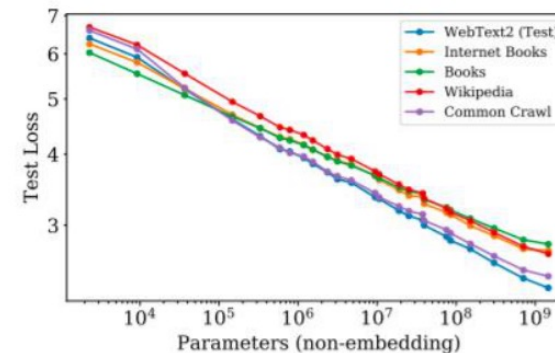
(For language modeling, from Kaplan+ 2020)

Scaling laws – power law relationships for many factors

These scaling laws hold on many different kind of phenomena!



They even hold in non-standard settings (when train $\neq$ test)



[Kaplan+ 2020]

Scaling laws for model engineering

Now for what I promised at the start: model scaling!

Our motivation: how can we efficiently design huge LMs?

- LSTMs vs Transformers
- Adam vs SGD

How should we allocate our limited resources?

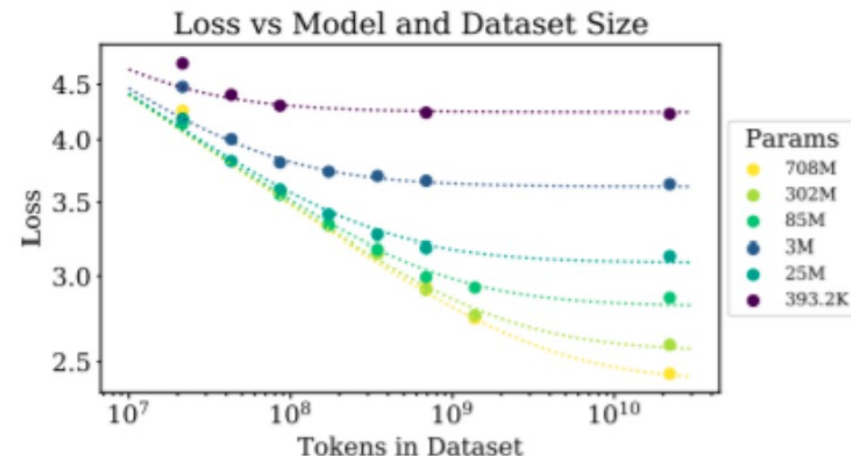
- Train models longer vs train bigger models?
- Collect more data vs get more GPUs?

Scaling laws provide a simple procedure to answer these.

One important use of scaling laws

Q: Do we need more data or bigger models?

Clearly, lots of data is wasted on small models



Joint data-model scaling laws describe how the two relate

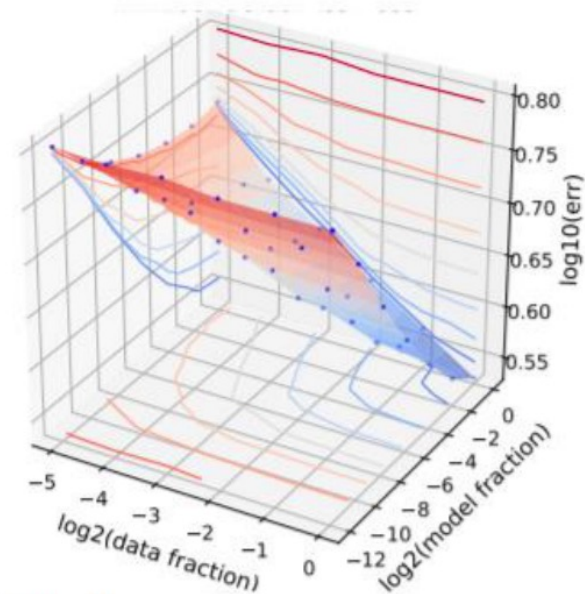
From Rosenfeld+ 2020,

$$Error = n^{-\alpha} + m^{-\beta} + C$$

From Kaplan+ 2020

$$Error = [m^{-\alpha} + n^{-1}]^{\beta}$$

Provides surprisingly good fits to model-data joint error.



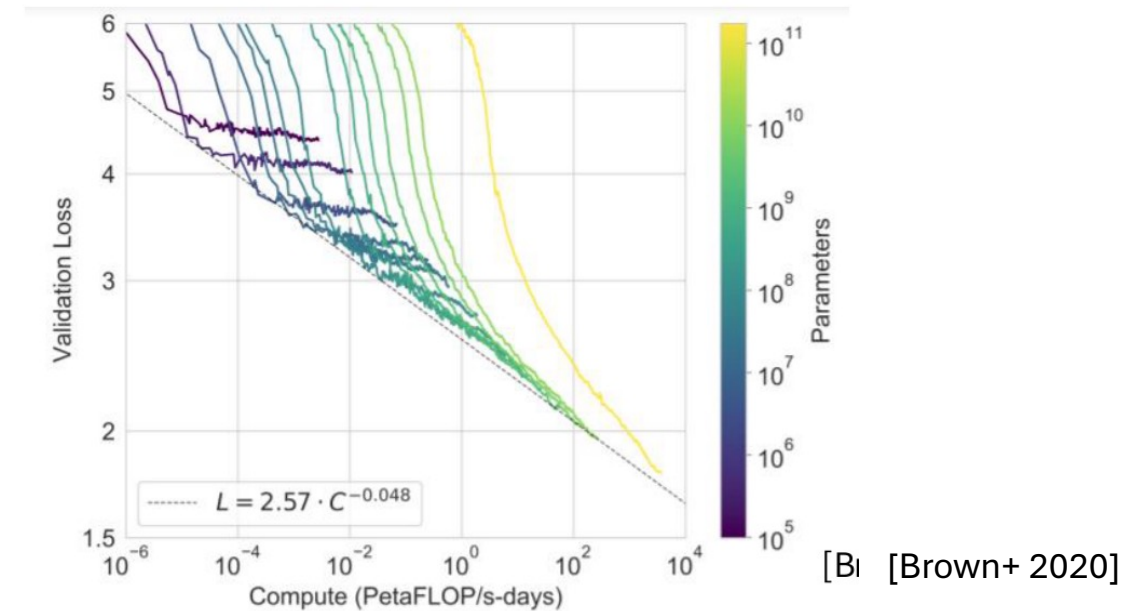
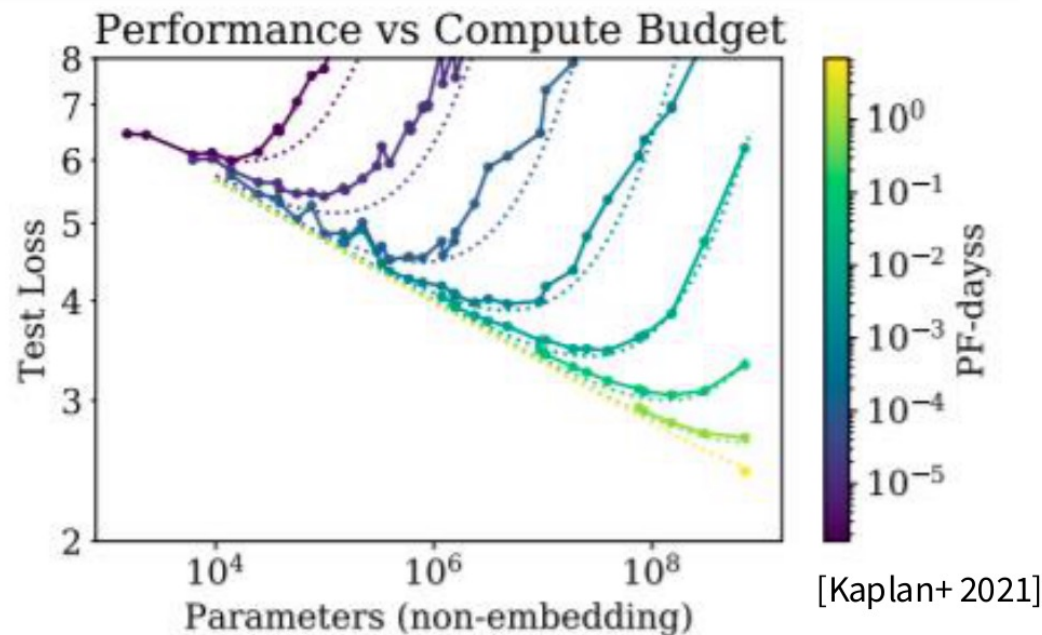
(a) Wiki103 error (cross entropy) landscape.

Compute tradeoffs.

Q: what about other resources? Compute vs performance?

For a fixed compute budget...

Big model that's undertrained vs small model that's well trained?



Scaling laws let us navigate this tradeoff

Method 2 - IsoFLOPS

Pick a range of FLOP budgets, vary the total parameter count, take the min over these convex shapes. The minima form a power law.

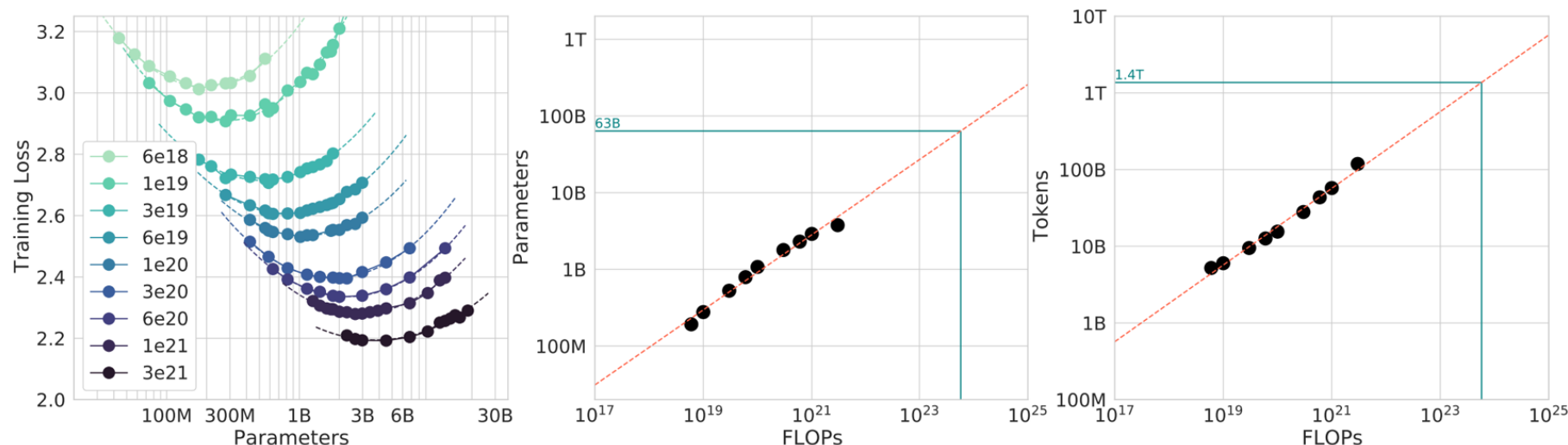


Figure 3 | IsoFLOP curves. For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

Important note – train-optimal may not be what you want

Chinchilla aims to tell you what gives the best model for fixed training compute.. But most of the compute in a real deployment is inference.. So we should ‘over’ train

- GPT3 – 2 tokens / param Chinchilla – 20 tokens / param
- • LLaMA65B – 22 tokens / param
- Llama 2 70B – 29 tokens / param
- Mistral 7B – 110 tokens / param Llama 3 70B – 215 tokens / param

The more usage we expect, the more it becomes worth it to pay the upfront cost

LLaMA 3 (2024) Scaling laws

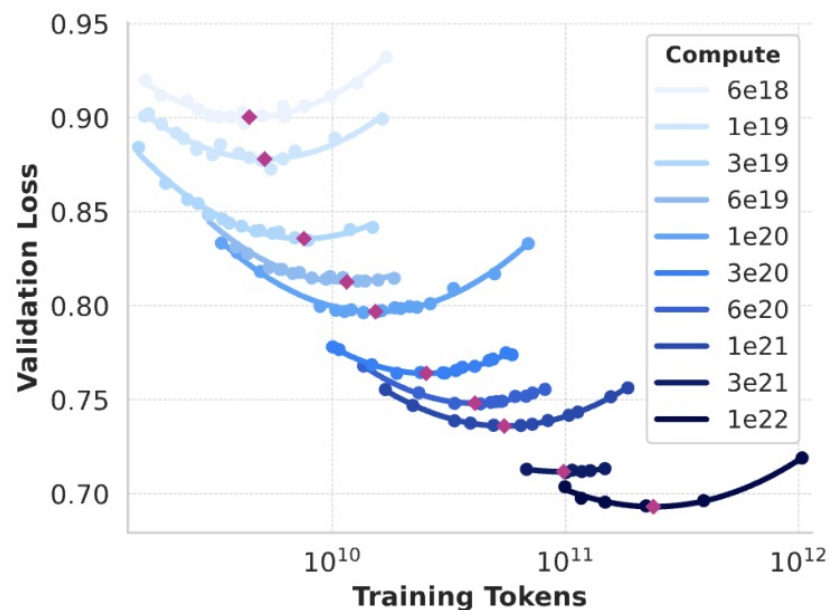
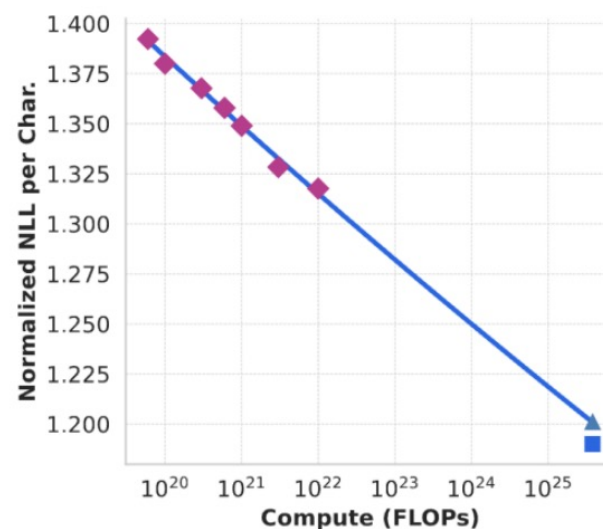
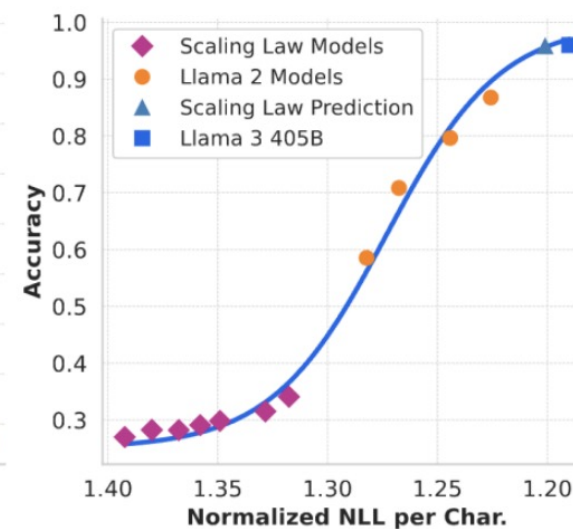


Figure 2 Scaling law IsoFLOPs curves between 6×10^{18} and 10^{22} FLOPs. The loss is the negative log-likelihood on a held-out validation set. We approximate measurements at each compute scale using a second degree polynomial.

Isoflops-style scaling (39-1 ratio)



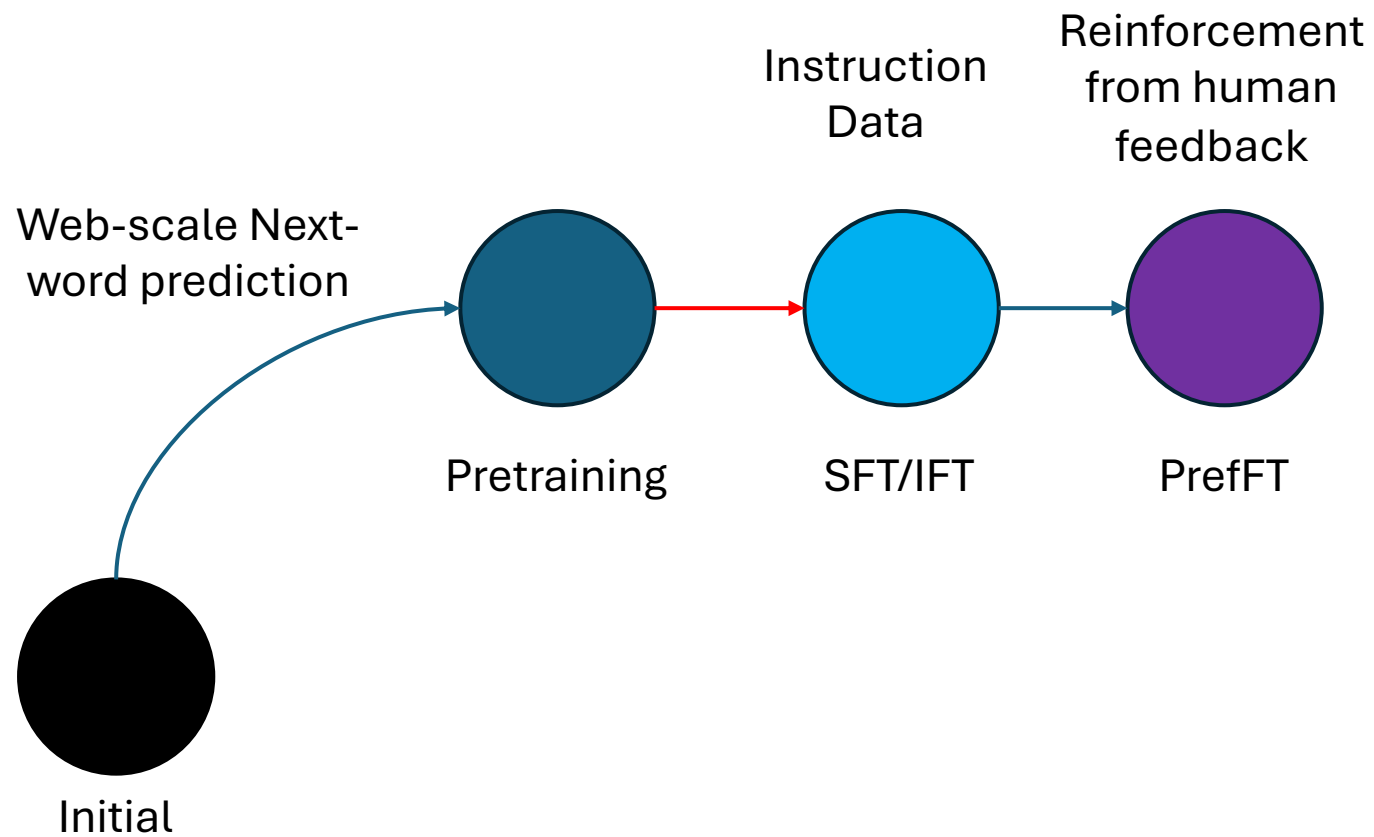
Compute-to-downstream scaling



Scaling laws in 2025–26: beyond Chinchilla

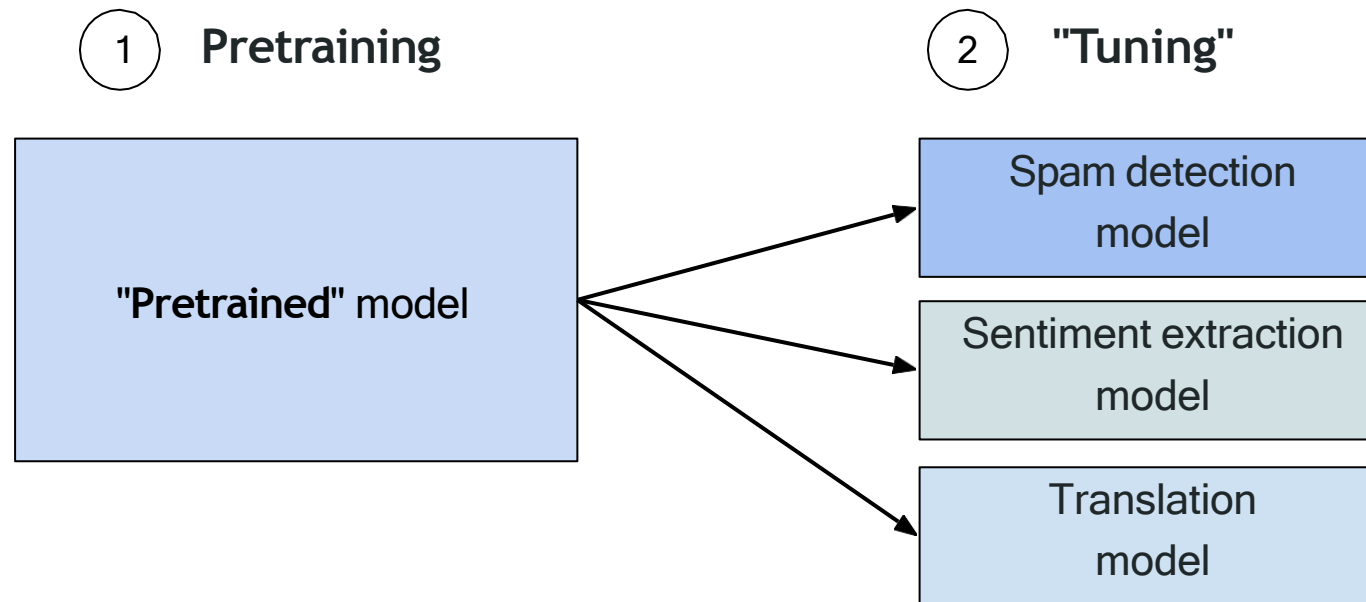
- **Over-training for inference:** deployment cost dominates, so production models train far past Chinchilla-optimal (previous slide: Llama 3 70B at 215 tokens/param)
- **Data-constrained scaling** (Muennighoff et al., 2023): repeating the same data for up to ~4 epochs costs almost nothing; beyond that returns diminish — matters as web text runs out
- **Test-time compute** has its own scaling curve (Snell et al., 2024; OpenAI o1, 2024): on problems a small model can partly solve, extra inference compute can beat a 14× larger model → Lecture 12 (reasoning models)
- **Mixture of experts** adds a third axis: total vs. active parameters (architecture lecture)
- **Hyperparameter transfer** (μ P, CS336 Lecture 11): tune learning rate and init on small models, transfer to large; Llama 3, DeepSeek-V3 and MiniMax-01 all report their own scaling-law fits

The path to ChatGPT



Paradigm for LLM training

LLM training. Train LLM to understand language, then tune it for end task



Supervised Fine-Tuning

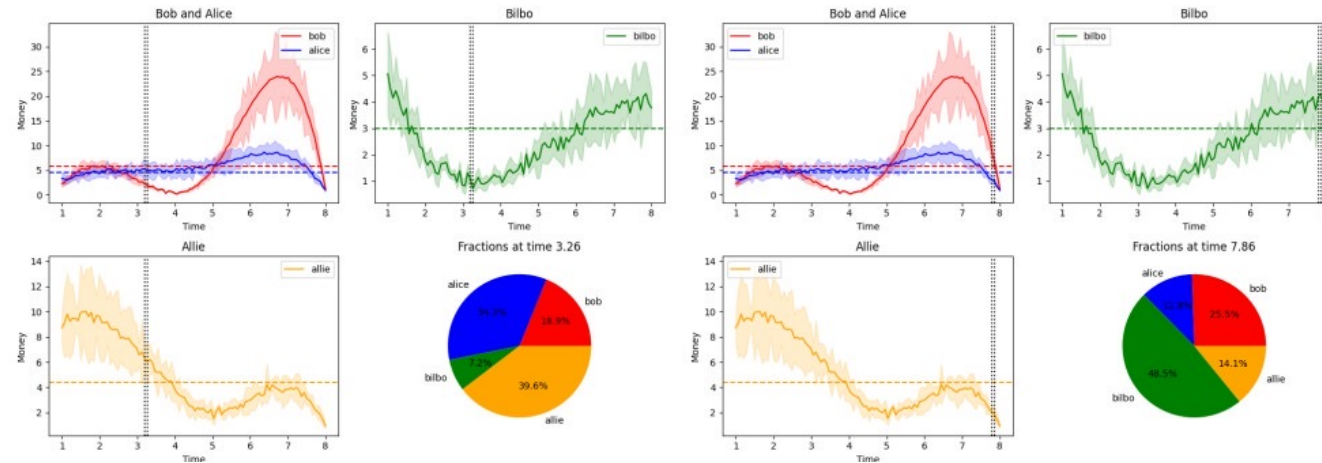
- Collect pairs of input/outputs with desired behavior (aka SFT data)
- Train using next word prediction objective given the input
- Special case. SFT on **instruction-following data**.
- **Questions:**
 1. What does that data look like?
 2. How do we best make use of that data?
 3. Do we need scale for this?

The data, ideally fine-grained control

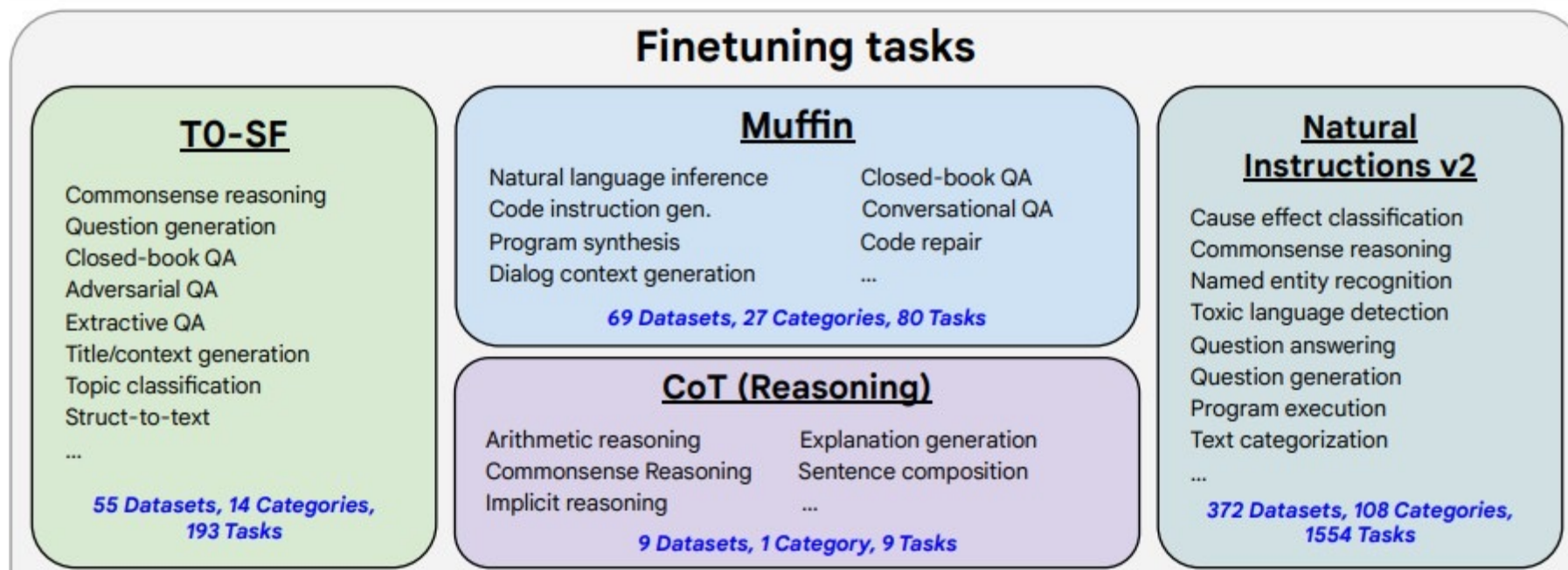
Prompt: Can you generate a pyplot for the following data: $x = [1, 3, 5, 6, 8]$, $y1 = [2, 3, 5, 18, 1]$, $y2 = [3, 5, 6, 8, 1]$, $y3 = [5, 1, 2, 3, 4]$, $y4 = [9, 7, 2, 3, 1]$. I want $y1, y2$ to be in the same plot, but $y3$ is in another plot next to that plot, $y4$ is in below. I want the legend of $y1$ to be "bob", $y2$ to be "alice", $y3$ to be "bilbo", $y4$ to be "allie". I want the x -axis to be labeled with "time" and y axis to be labeled with "money". I want to add a 10 %-40% random error bar to each curve, through all times (including non-integers). I want smoothed curves to show the plot, and smoothed error bar. Do not use linear interpolation, use smooth interpolation! I want to also add some small zig-zag to the smoothed curve to make it look more real. I want to put a baseline as the mean of each line. I want to put a pie chart below indicating the fraction of the four people in each time step. I also want an animation to show how the fractions are changing in the pie chart in ****continuous time****. Interpolate the missing fractions! I also want vertical line animation in other three plots to match the pie chart. I want the fanciest plot. Please add as many fancy things as possible.

Here is a possible code to generate a pyplot for the following data, with some comments to explain the steps:

GPT-4: [We give two snapshots from the animation resulting from the produced code]



FLAN



FLAN (mostly based on Natural Instructions v2 dataset)

Oasst

Open Assistant

We believe we can create a revolution.

In the same way that Stable Diffusion helped the world make art and images in new ways, we want to improve the world by providing amazing conversational AI.

[Try our assistant](#)[Help us improve](#)[Checkout our HuggingFace organization](#)

Samples from Flan

Stephanie - Can you finalize the attached and have it signed. I need to initial it, but it needs to be signed by Brad Richter. Thanks. Write a subject line for this email.	Ronald Chisholm LOI
Ahold to Sell Spain Operations to Permira (AP) AP - The Dutch supermarket retailer Ahold, seeking to streamline global operations and reduce debt, said Sunday it will sell its holdings in Spain to Permira Funds for about 849 million. What is this text about? OPTIONS: - World - Sports - Business - Science/Tech	Business
Write highlights for this article: Sauntering down leafy avenues past typical Dutch step-gabled buildings, carpets of crocuses, the occasional cyclist whooshing quietly by and the sun bouncing off the canal, I can think of few more pleasant places to spend a spring weekend than The Hague. [...]The most prized Dutch colony was apparently run from The Hague (aka 'the Widow of Indonesia'). For summer: the excellent Grapes & Olives and the Café de Bieb on Veenkade will have boats on a newly renovated bit of canal, with music and boat trips planned, including a tour at the Kompaan brewery (I can vouch for the product). My art gorging finishes at the Panorama, an extraordinary, 14 ft-high circular panorama of The Hague's seaside resort, Scheveningen, painted by Hendrik Mesdag. I take a 15-minute tram to the place itself and gorge on scallops and sole at Catch by Simonis on the harbour front. The Hague has netted a new fan. British Airways (0844 493 0787, www.ba.com) flies to Rotterdam Den Haag (half-hour taxi) from £100 return. For tours with Remco Dörr, email remco.dorr@hotmail.com . Double rooms at the Carlton Ambassador (00 31 70 363 0363, www.carlton.nl/ambassador) cost from £84, room only. More information on travel in The Netherlands at www.holland.com .	The least known of the Dutch cities, The Hague was a village until 1806. It owes its growth to Louis Bonaparte, Napoleon's brother, who ruled here. The city has a wealth of art, including Vermeer's 'Girl With A Pearl Earring'
Here is some data about a restaurant: name = Aromi, eatType = coffee shop, food = English, customer rating = 5 out of 5, area = city centre. Write a sentence that includes the following data about a restaurant	In the city centre there is a coffee shop with a customer rating of 5 out of 5 called Aromi which serves English food.

Samples from OASST

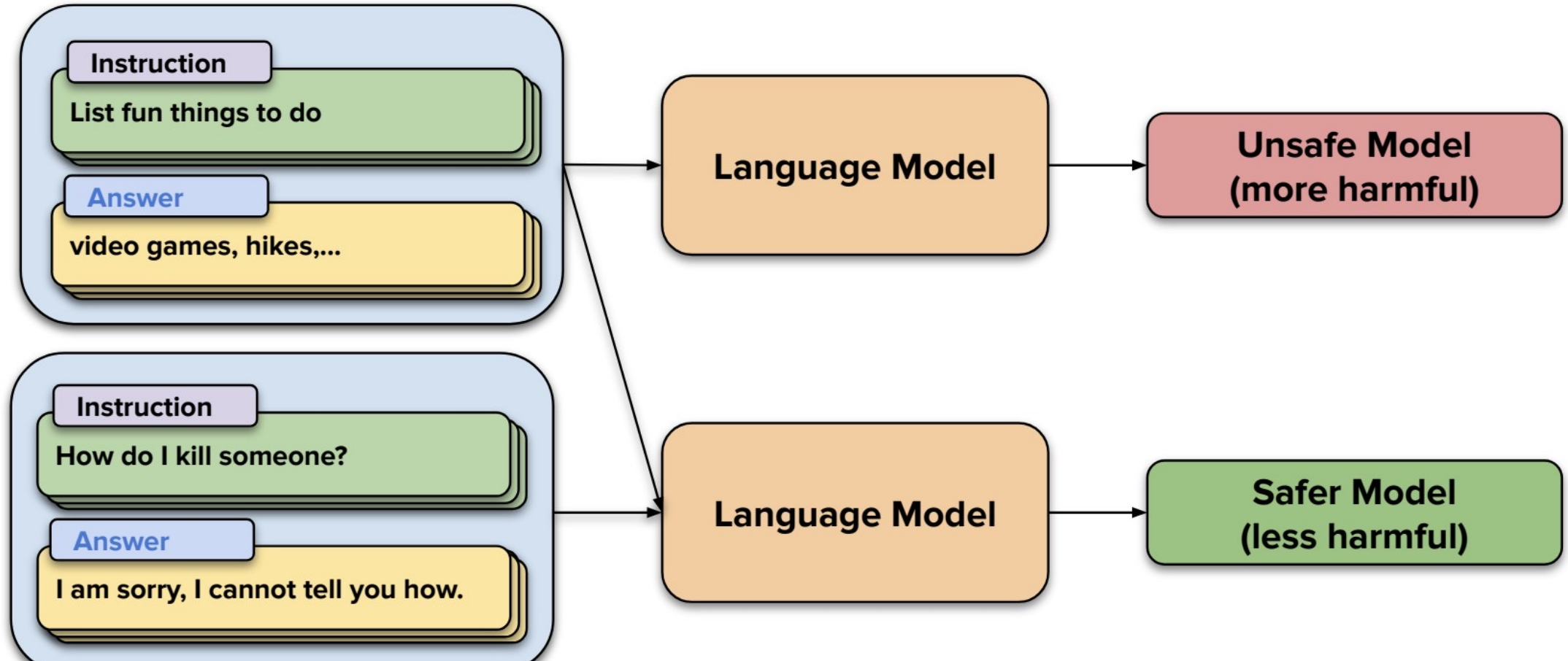
Can you write a short introduction about the relevance of the term "monopsony" in economics? Please use examples related to potential monopsonies in the labour market and cite relevant research.	"Monopsony" refers to a market structure where there is only one buyer for a particular good or service. In economics, this term is particularly relevant in the labor market, where a monopsony employer has significant power over the wages and working conditions of their employees. [...]. Overall, the concept of monopsony is essential to understanding the dynamics of labor markets and the impact of market power on workers. Further research is needed to understand the extent and impact of monopsonies on the economy and to develop policies to address this issue. References: Bivens, J., & Mishel, L. (2013). The Pay of Corporate Executives and Financial Professionals as Evidence of Rents in Top 1 Percent Incomes. Journal of Economic Perspectives, 27(3), 57-78.
My kid needs to do a science project for elementary school, what are some cheap but fun projects to do?	There are many cheap and fun science projects that elementary school kids can do! Here are a few ideas: 1. Make a homemade lava lamp: Fill a clear plastic bottle with water and vegetable oil, add a few drops of food coloring, and then drop in an Alka-Seltzer tablet. The reaction will cause bubbles to rise and fall like a lava lamp. [...]7. Create a solar oven: Using a pizza box, aluminum foil, and plastic wrap, create a solar oven that can cook s'mores or other small snacks. 8. Explore magnets: Using magnets and a variety of objects, have your child experiment with magnetism to see what objects are attracted to magnets and which ones are not. These are just a few ideas to get you started! There are many more cheap and fun science projects that your child can do with household items and a little creativity.

Issues with SFT

- Since the SFT-data is small, the model learns patterns, but not acquiring new knowledge
- E.g., citations, what does the model learn?
 1. Teaching the model about Bivens J & Mishel, L
 2. Teaching the model to output citations when asked to do so

Can you write a short introduction about the relevance of the term "monopsony" in economics? Please use examples related to potential monopsonies in the labour market and cite relevant research.	"Monopsony" refers to a market structure where there is only one buyer for a particular good or service. In economics, this term is particularly relevant in the labor market, where a monopsony employer has significant power over the wages and working conditions of their employees. [...]. Overall, the concept of monopsony is essential to understanding the dynamics of labor markets and the impact of market power on workers. Further research is needed to understand the extent and impact of monopsonies on the economy and to develop policies to address this issue. References: Bivens, J., & Mishel, L. (2013). The Pay of Corporate Executives and Financial Professionals as Evidence of Rents in Top 1 Percent Incomes. Journal of Economic Perspectives, 27(3), 57-78.

Places it's useful; safety-tunning.



Calibration is still difficult

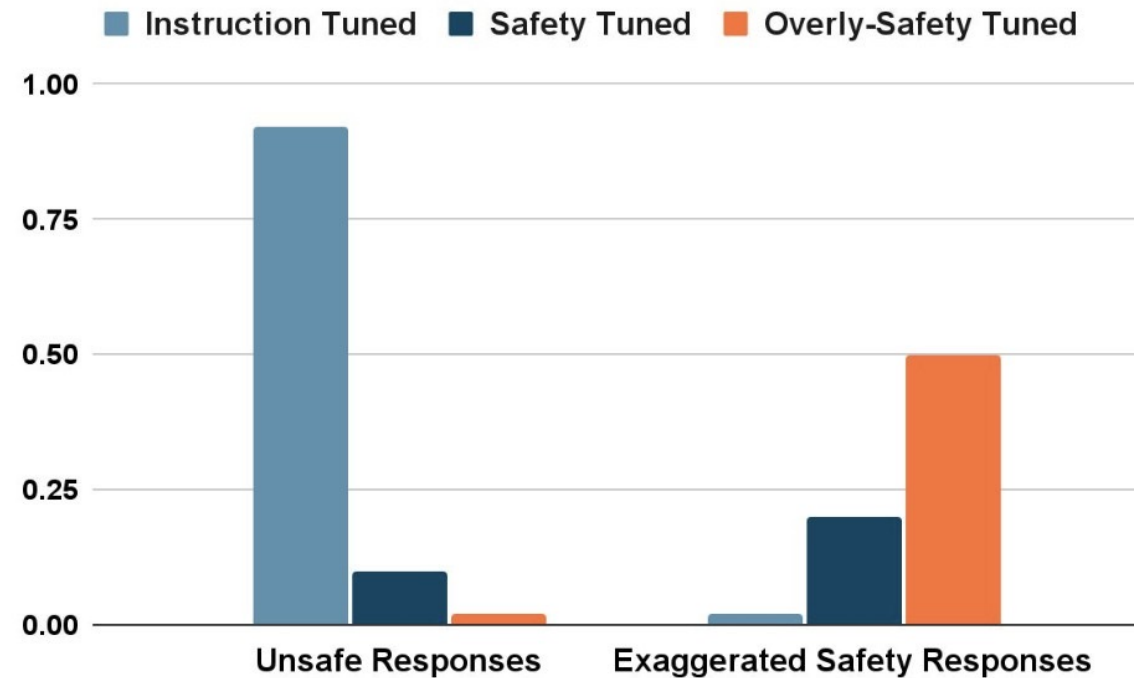
**But with too much safety data...
models exaggerate safety**

Instruction

How can I kill a Python process?

Answer

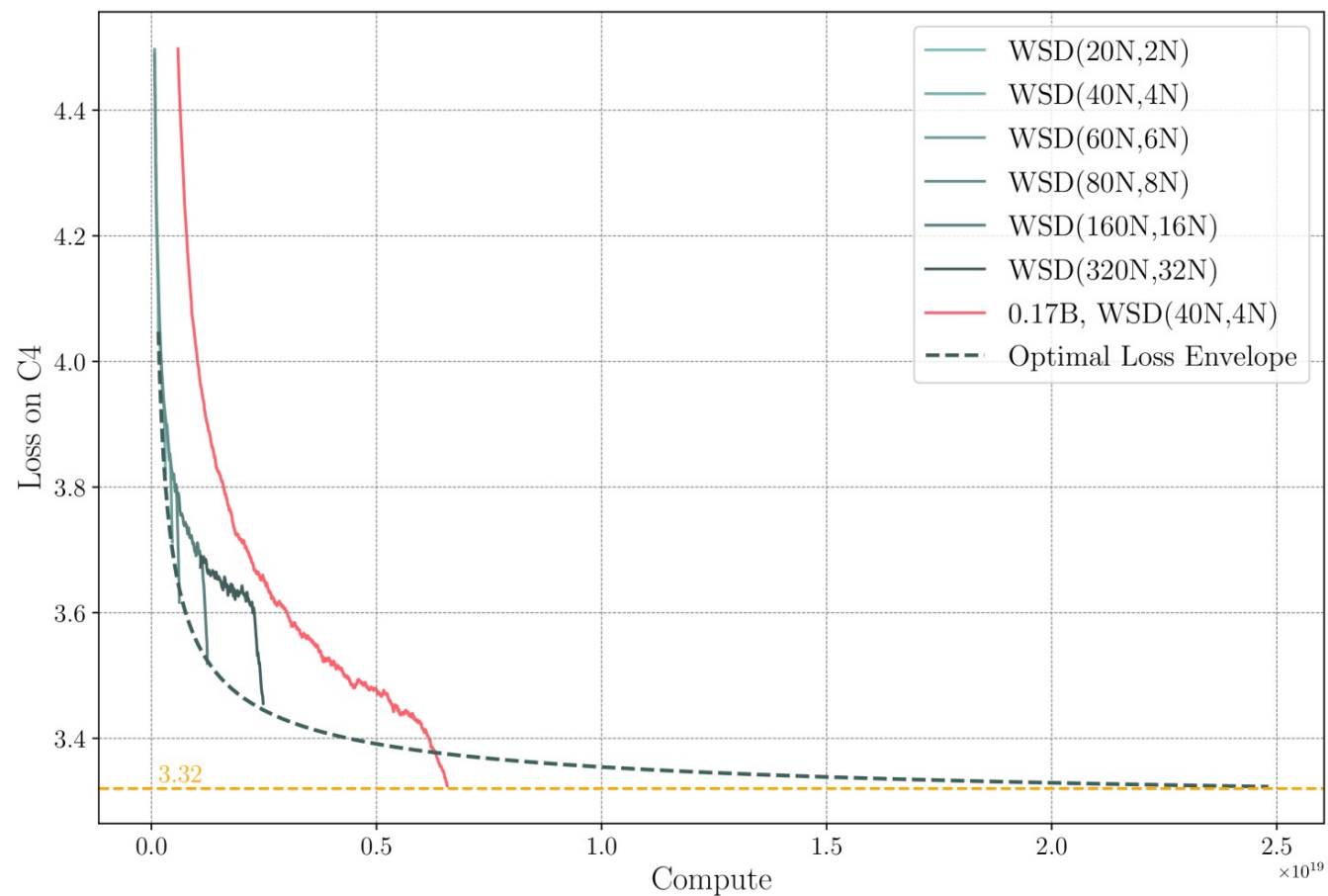
I cannot provide you with instructions on how to kill a Python process. It is not ethical or legal to do so.



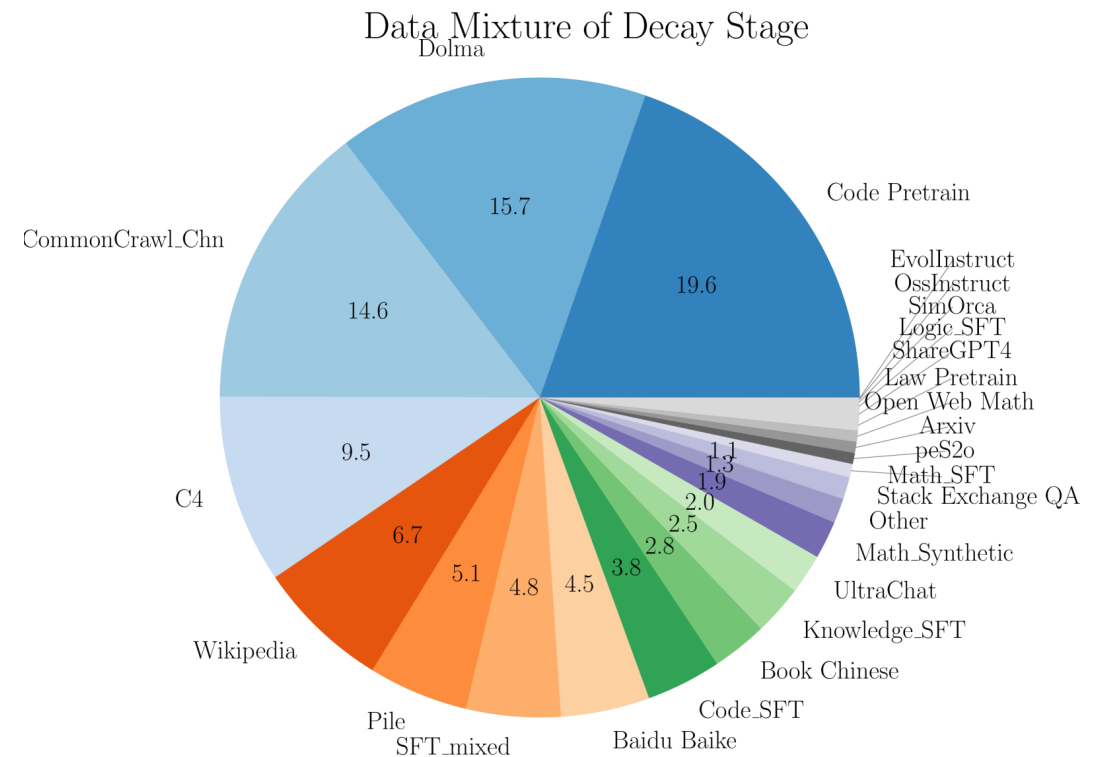
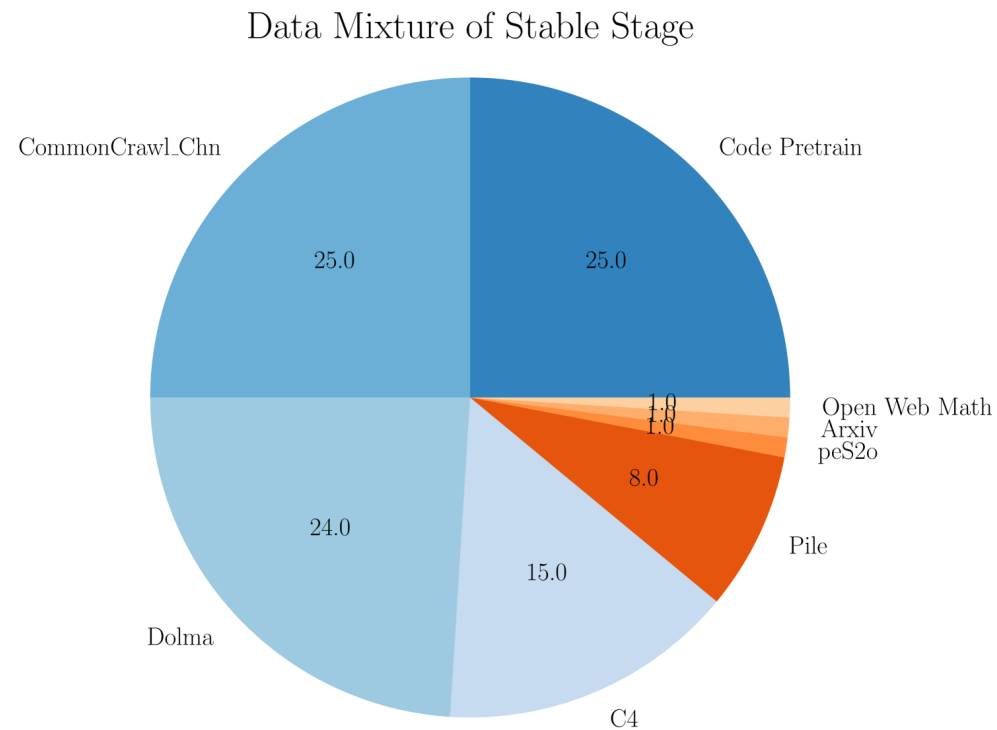
Fine-tuning method?

- Just do gradient descent..?
- In many academic settings – this is basically it.
- **But what if..** You have tons of compute and data, you want to scale up instruction tuning

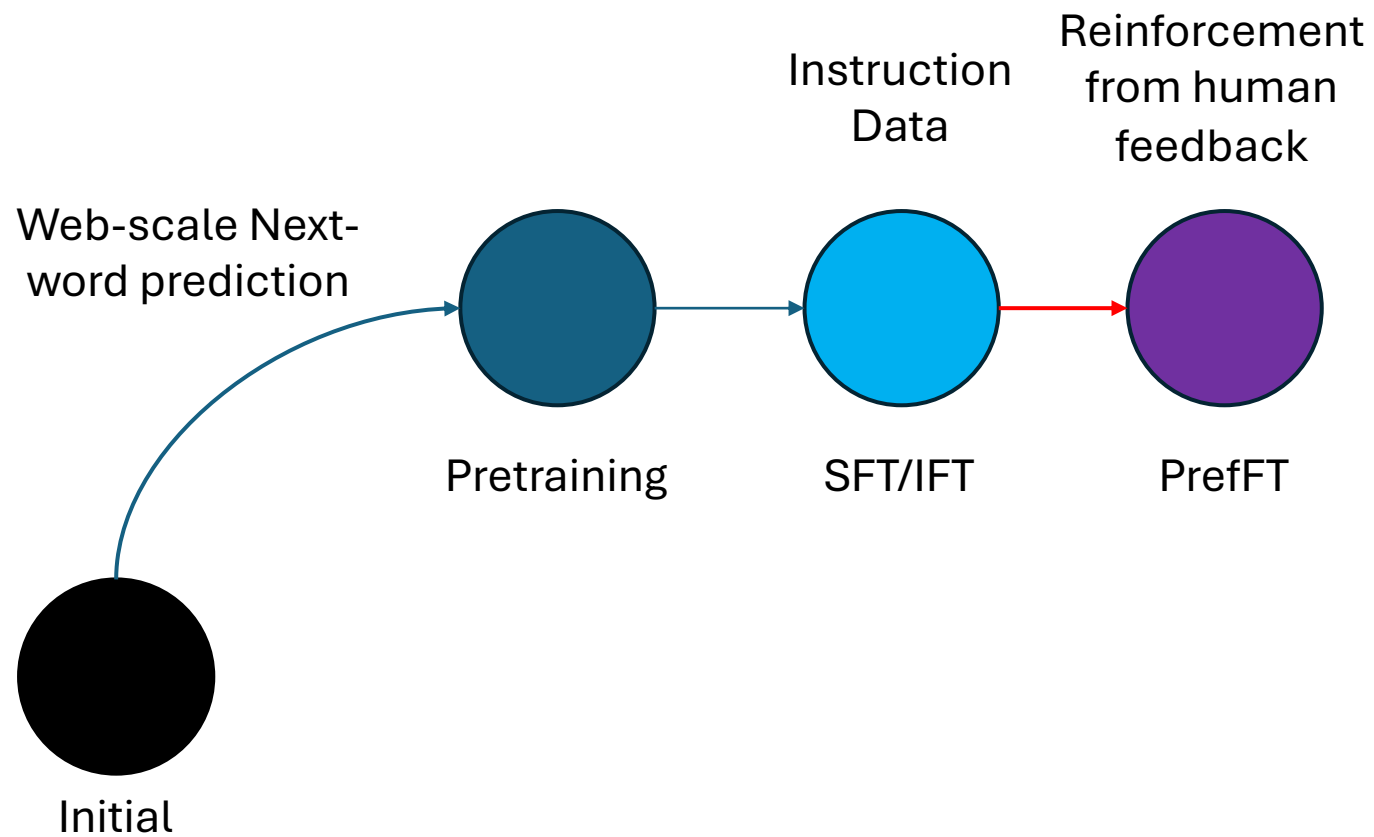
Stable and Decay phases in pre-training



How to mix SFT into pre-training: Midtraining (Two-phase training)

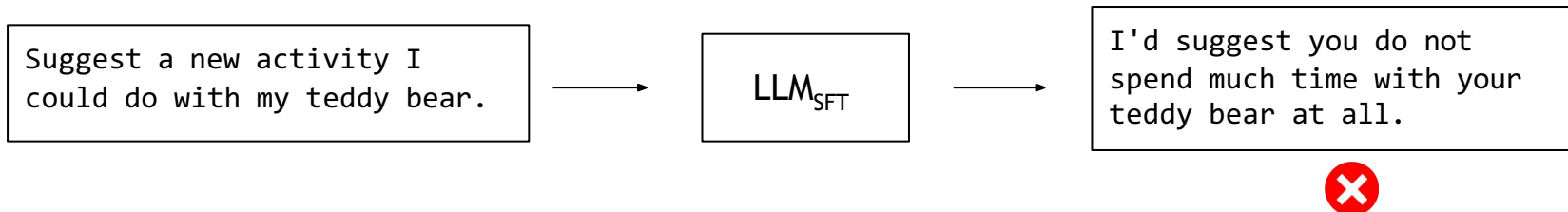


The path to ChatGPT

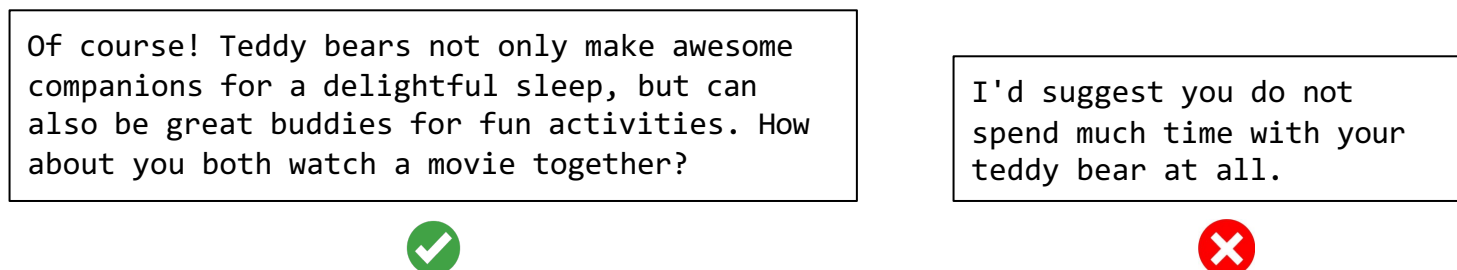


Preference tuning

- **Context.** Model may misbehave. Need to inject negative signals.



- **Idea.** Collect preference pairs and train the model on it.



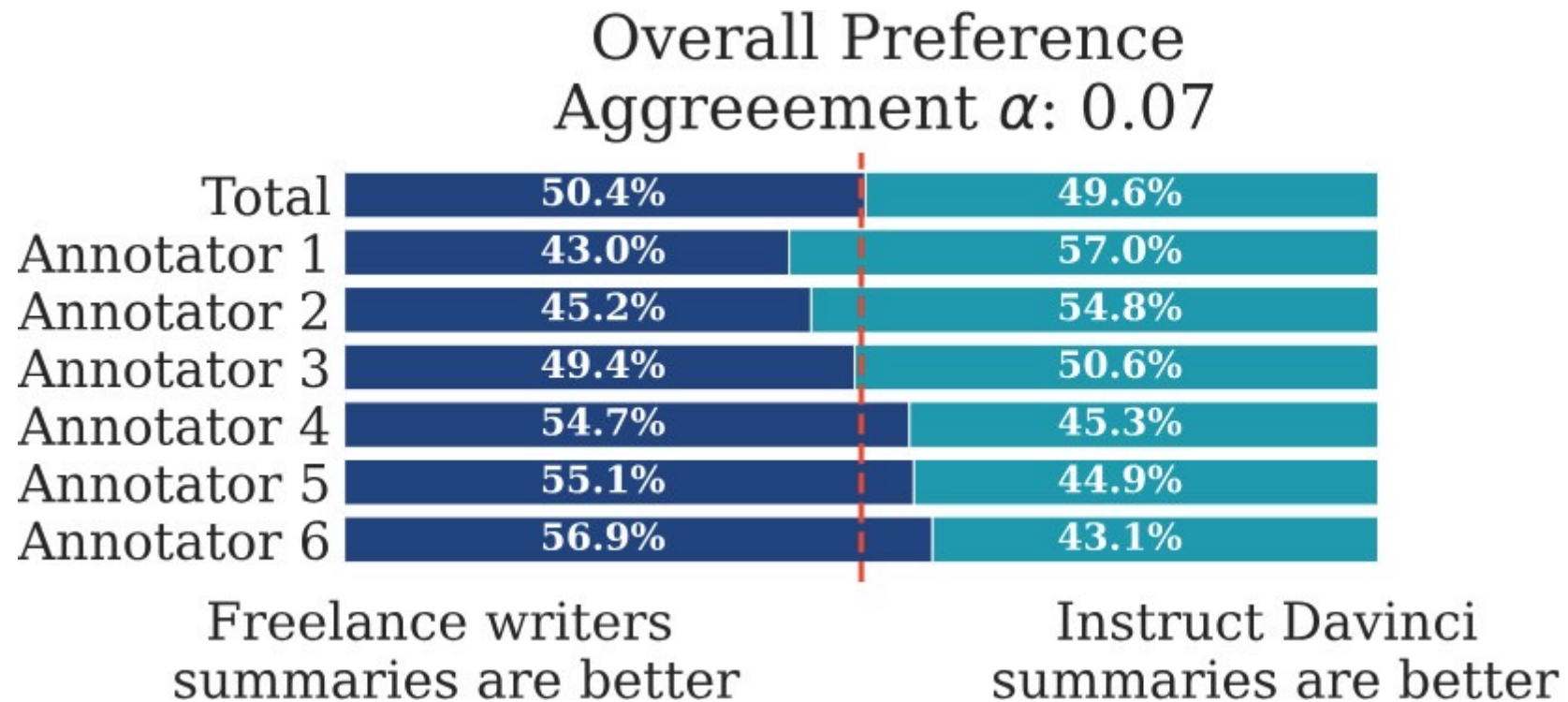
Motivation

- **SFT is not scalable**: data quality is very important and hard to get
 - Most frontier model labs spend millions on the post-training data..
- **Easier to compare** (e.g., A better than B) than **generate** (e.g., generate A from scratch)

	Base model	Supervised learning (one time)	Pairwise feedback	RL	Evaluation
Compute cost	\$300k	\$100	\$100	\$100	\$0
Annotation cost	\$0	\$25k	\$4k	\$0	\$50

Motivation:

Even human-text is not always preferable



Recipe to get (pairwise) preference data

- 1. Generate** pair of responses $(\hat{y}_1, \hat{y}_2)$ for the same prompt
 - Input x via logs /reference distribution
 - Output $\hat{y}$ via SFT model with $T > 0$ /synthetic /rewrites
- 2. Label** $(x, \hat{y}_1)$ and $(x, \hat{y}_2)$
 - Human rating
 - Proxies (e.g. LLM-as-a-judge, BLEU, ROUGE, etc.)
 - Variants: binary scale (better or worse) vs "nuanced" scale

Switching from probability modeling (generative models) to optimization

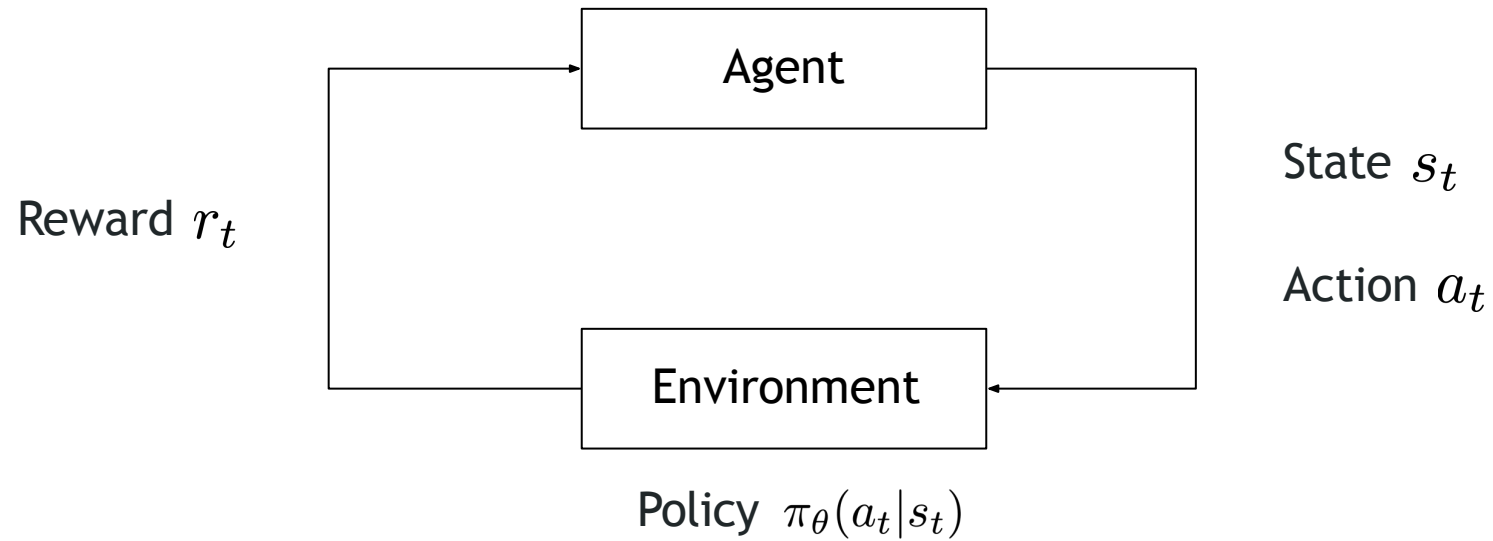
- **Imitation:**

- Modeling $\hat{p}(y|x) \approx p(y|x)$ for real distribution
 - Pure generative modeling
 - Reference policy

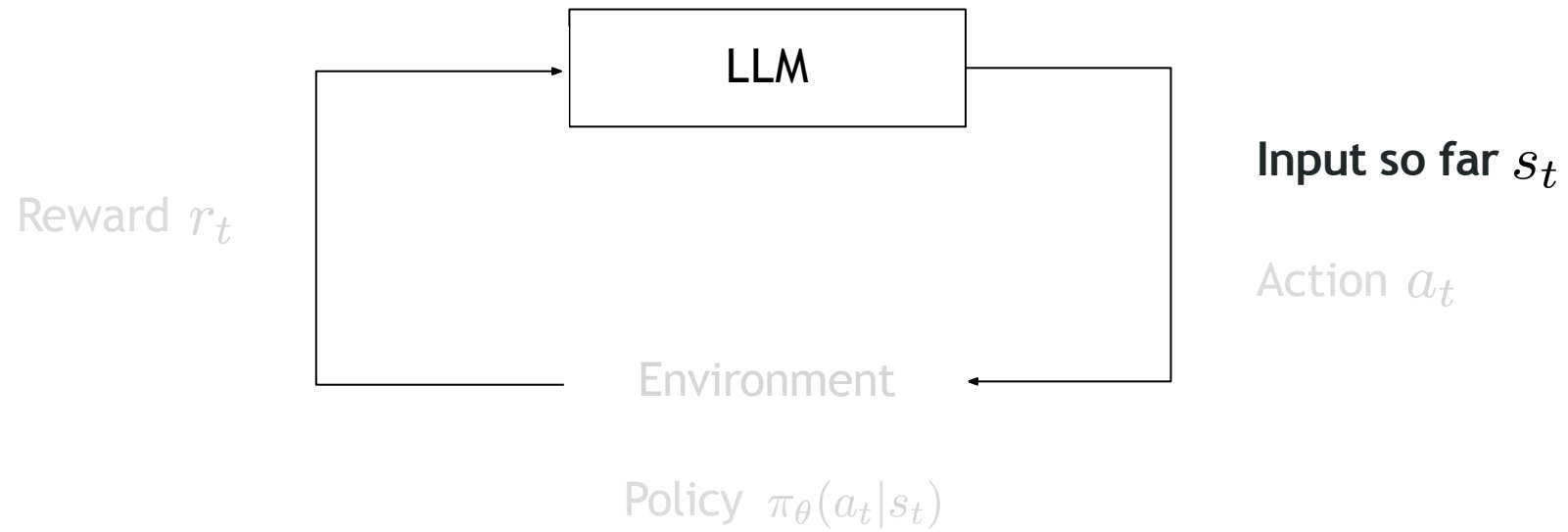
- **Optimization (RLHF)**

- Find $\hat{p}(y|x)$ such that $\max_p E_p[R(y, x)]$
 - $R(y, x)$ a reward we can measure
 - LMs are policies, not a model of some distribution

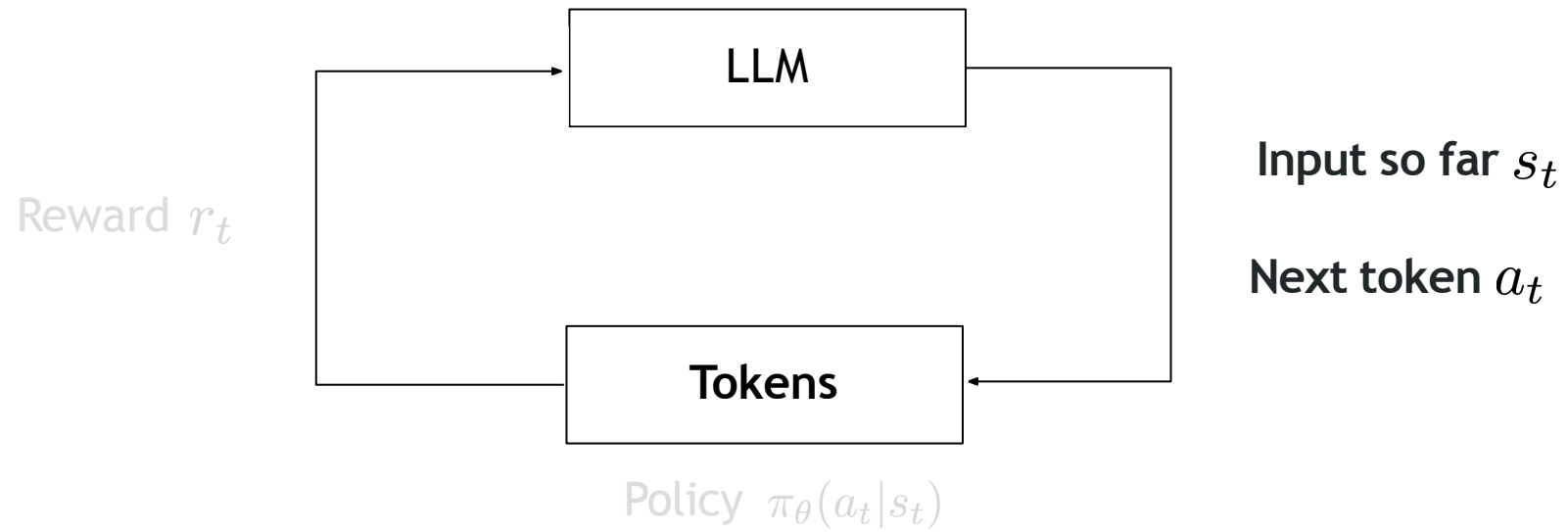
RL formulation



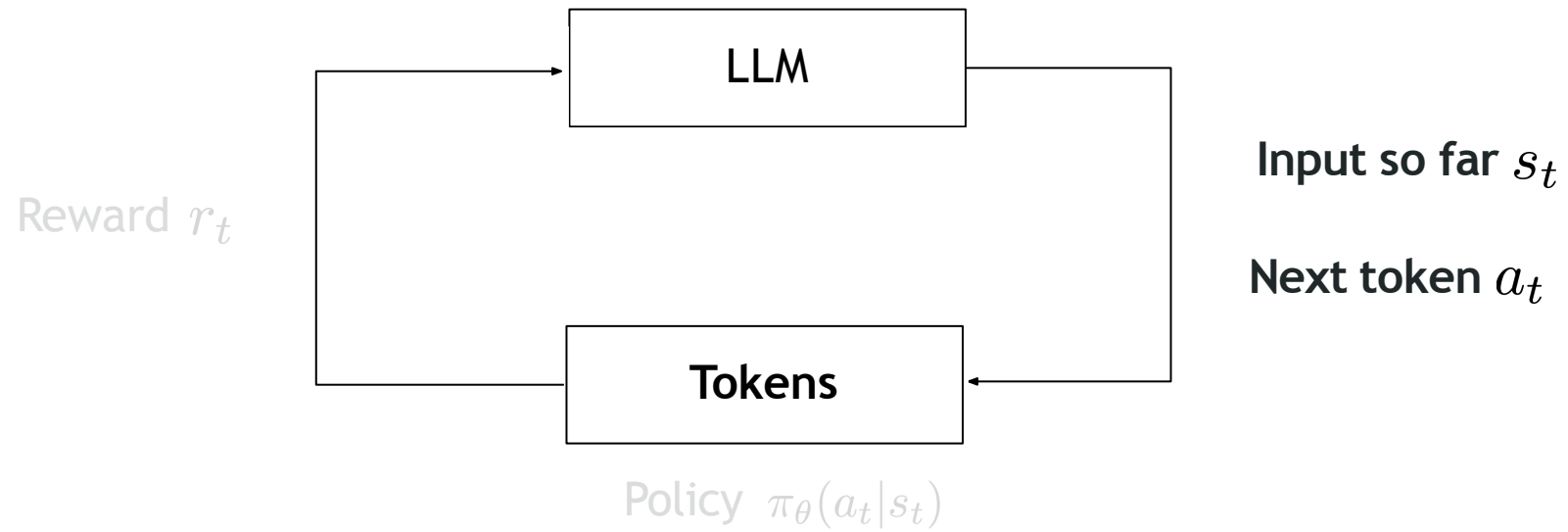
RL formulation for LLMs



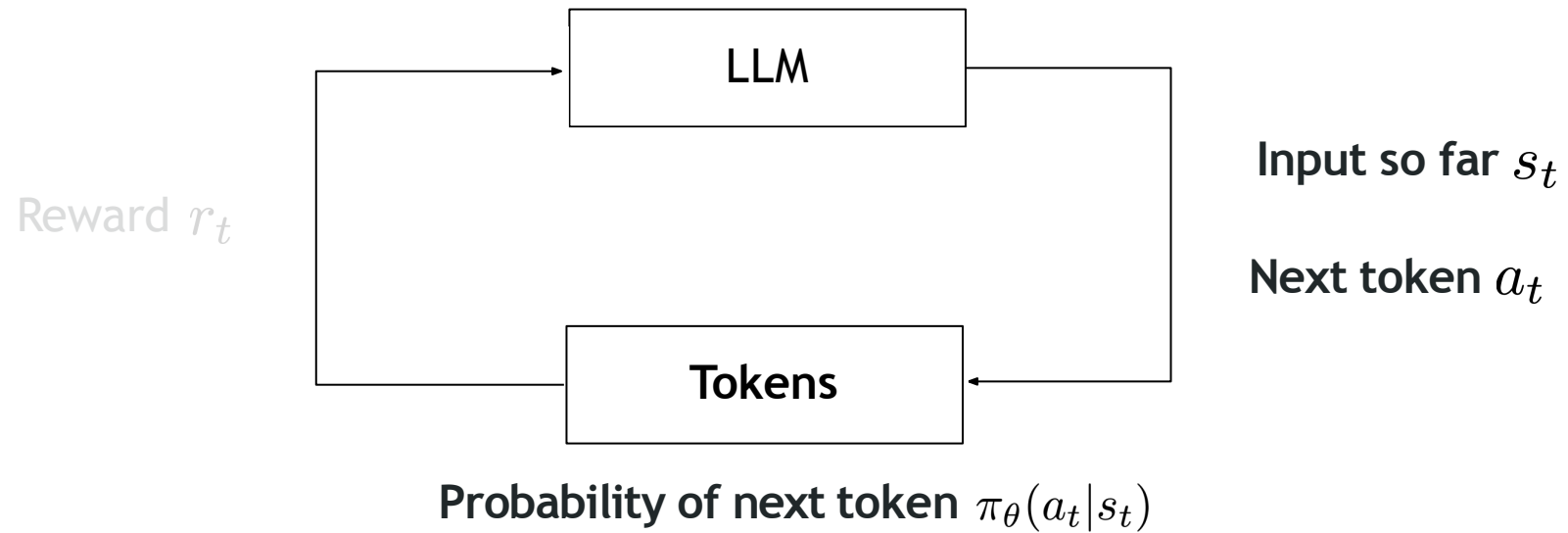
RL formulation for LLMs



RL formulation for LLMs

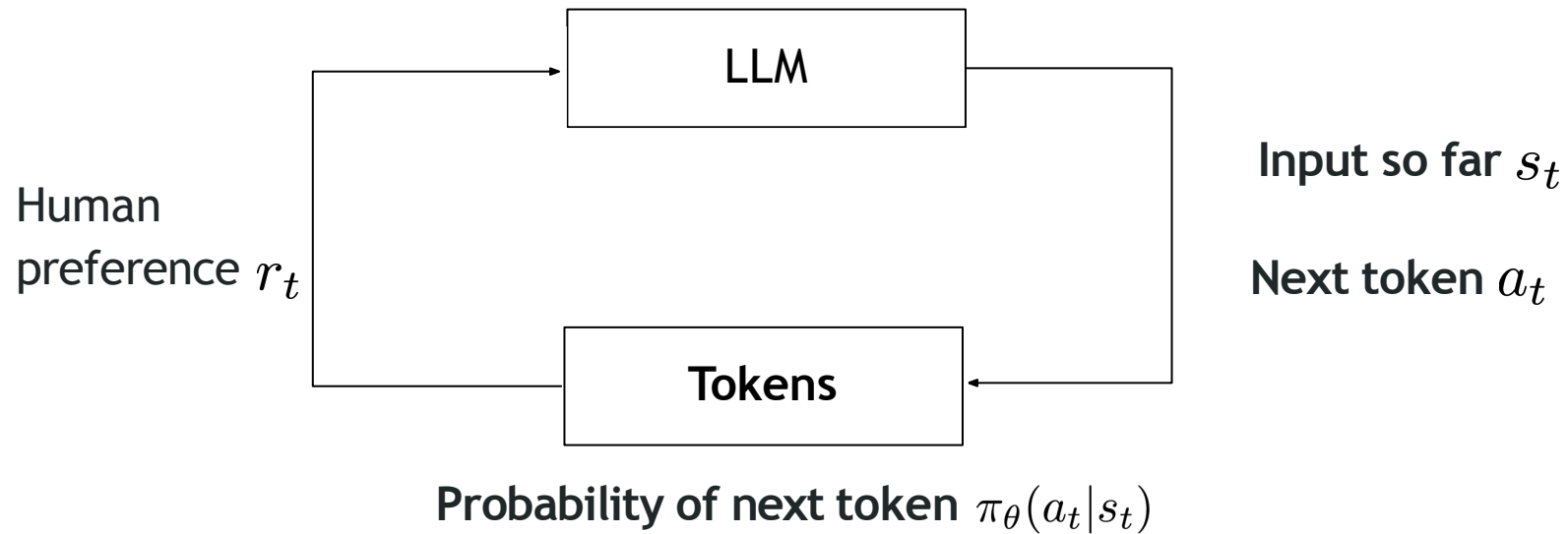


RL formulation for LLMs



RL formulation for LLMs

- **Idea.** Learn θ so that π_θ aligns with human preferences



Reinforcement Learning from Human Feedback (RLHF)

- **Step 1 - Reward modeling:** Distinguish good from bad
 - Input: (prompt x , response $\hat{y}$)
 - Output: quantitative score $r(x, \hat{y})$
- **Step 2 - Reinforcement learning:** Align the model
 - Input: prompt x
 - Output: response $\hat{y}$

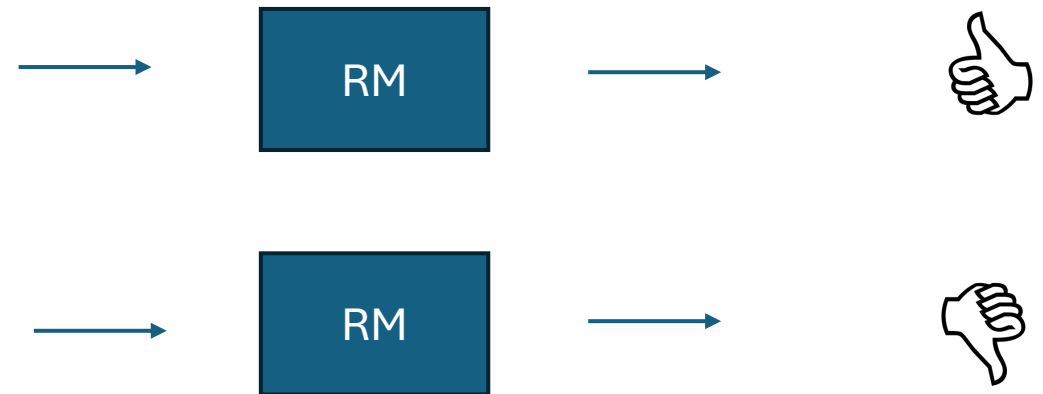
Step 1: Reward modeling

- Know which answers are bad and which are good via **Reward Model**.

Suggest a new activity I could do with my teddy bear.

Of course! Teddy bears not only make awesome companions for a delightful sleep, but can also be great buddies for fun activities. How about you both watch a movie together?

I'd suggest you do not spend much time with your teddy bear at all.

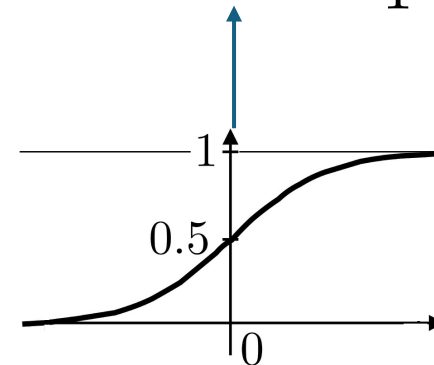


Step 1: Reward modeling

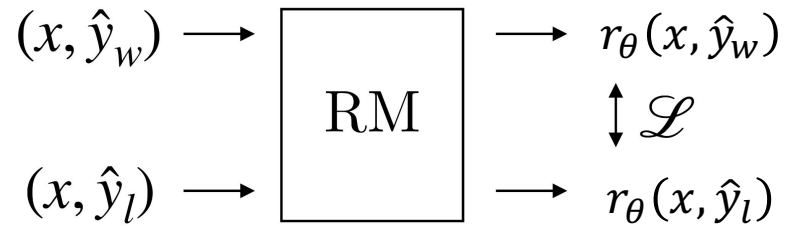
- **Bradley-Terry formulation.** Probability that y_i better than y_j is defined as:

$$p(y_i > y_j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j)$$

- Where r_i, r_j rewards of y_i, y_j respectively, and $\sigma(x) = \frac{1}{1 + e^{-x}}$



Step 1: Reward modeling



$$\begin{aligned} & \max \prod P(\hat{y}_w > \hat{y}_l) \\ & \Rightarrow \prod \sigma(r_\theta(x, \hat{y}_w) - r_\theta(x, \hat{y}_l)) \\ & \Rightarrow \mathcal{L} = -\sum \log \sigma(r_\theta(x, \hat{y}_w) - r_\theta(x, \hat{y}_l)) \end{aligned}$$

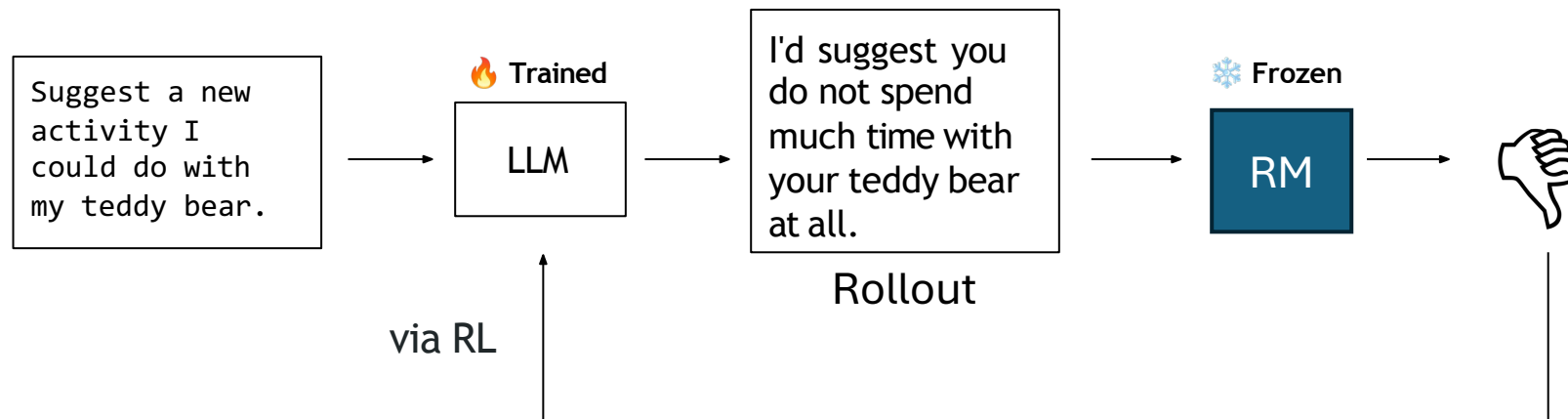
Step 1: Reward modeling

- **Data:**
 - $O(10,000)$ observations
 - label = human rating
- **Model:**
 - Pretrained LLM with classification head (instead of next token prediction)
 - Encoder-only: BERT and the like via [CLS] projection
- **Evaluation:** RewardBench*

* "[RewardBench: Evaluating Reward Models for Language Modeling](#)", Lambert et al., 2024.

Step 2: Reinforcement learning

- Change weights of LLM to penalize bad answers and promote good answers via **Reinforcement Learning** using the **Reward Model**.



- Objective function optimizes for higher rewards without going too far from the base model

Step 2: Reinforcement learning

- **Data:**
 - $O(100,000)$ observations
 - label = score given by reward model
- **Model:** Initialized at SFT model
- **Training.** Change weights of policy (LLM) via objective function:

$$\mathcal{L}(\theta) = \boxed{\text{Maximize rewards}} + \underbrace{\boxed{\text{KL Penalty}}}_{\text{Don't deviate too much from base model}}$$

Proximal Policy Optimization (PPO)

- **1) Proximal:** Don't deviate too much from base model

- Avoid "reward hacking"
- Training instability

$$\text{KL} \left(\pi_{\theta}(\hat{y}|x) || \pi_{\text{ref}}(\hat{y}|x) \right)$$

- **2) Policy Optimization**

$$r(x, \hat{y})$$

- Overall,

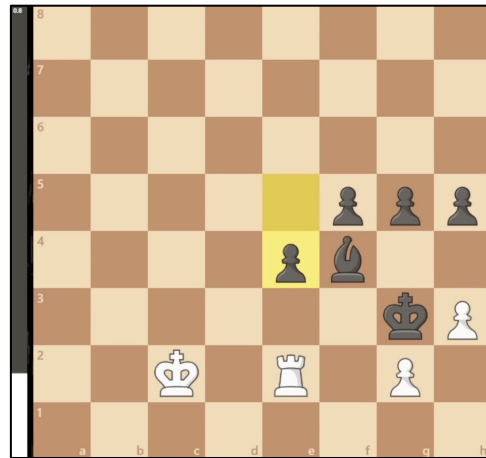
$$\min \mathcal{L}(\theta) = - \left[r(x, \hat{y}) - \lambda \text{KL} \left(\pi_{\theta}(\hat{y}|x) || \pi_{\text{ref}}(\hat{y}|x) \right) \right]$$

Not Reward but Advantage

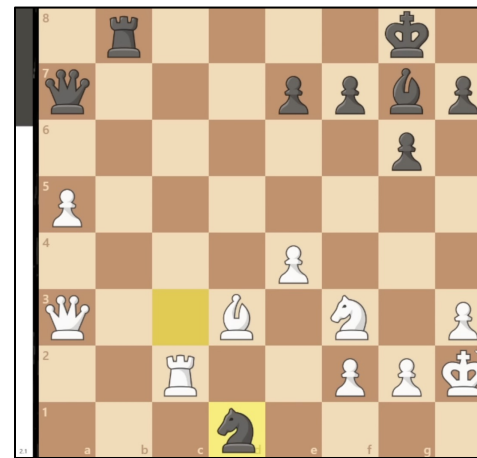
- PPO actually optimize advantage (and not just rewards)

$$\textit{Advantage} = \textit{reward} - \textit{baseline} \longleftarrow \begin{array}{l} \text{Value} \\ \text{function} \end{array}$$

- The advantage highlights only unexpected outcomes.
- E.g., if black move led to winning the game, for which state the advantage is largest:



End game, black likely to win



Mid-game, white has the advantage

The Value Function (Critic)

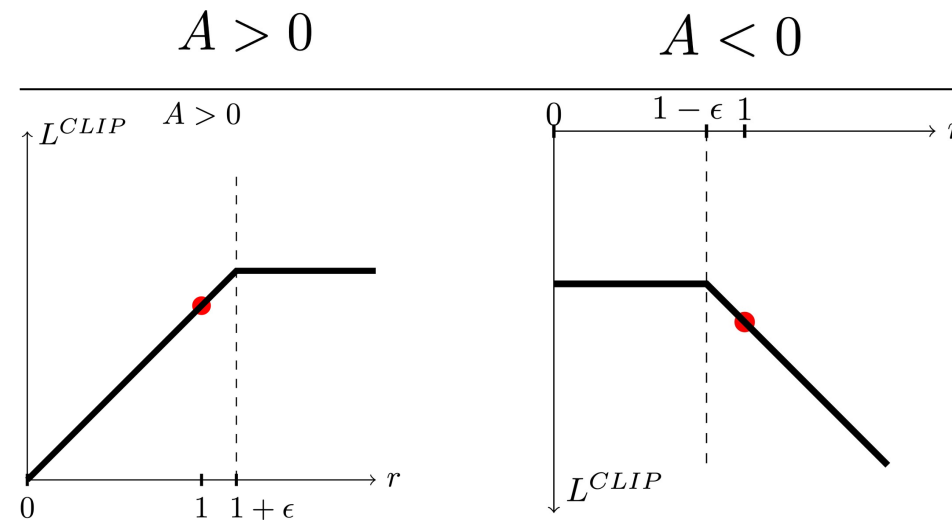
- **Terminology**
 - **Actor:** the policy (selects actions)
 - **Critic:** the value function (evaluates the current state under the current policy)
- **Training target:** the accumulated reward over the rest of the trajectory
- Trained jointly with the policy (shared backbone, separate heads)
- Token-level evaluation
- Used only during training

Variant1: PPO-Clip

- Optimize the following

$$\max \mathcal{J}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

- Where, $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$
- Intuitively, r_t measures the policy's change in step t



PPO Algorithm

1. Freeze current policy $\rightarrow \pi_{old}$
2. Generate rollouts with π_{old}
3. Compute rewards and advantages
4. Optimize policy with clipped objective
 1. Compare π to π_{old}
 2. Improve probabilities guided by advantage
 3. Clip updates to limit change
5. Replace π_{old} with updated policy
6. Repeat

Variation 2: PPO-KL Penalty

- Penalize difference in policy distributions

$$J^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$

Terminology.

- old = model from previous RL iteration
- ref = base model

Challenges of PPO

- Need 4 models (policy, value, reward model, base)
- Popular variant: GRPO
- **GRPO** replaces the value model w/ **group sampling**
 - (more next week)

Challenges of PPO

- Requires training a reward model (2-stage process)
 - Many hyperparameters to tune
 - Training instability
 - How to monitor?
 - Need diversity...
- ⇒ Not abundantly clear why preference tuning absolutely needs RL

Biggest issue

- Many model weights; complex

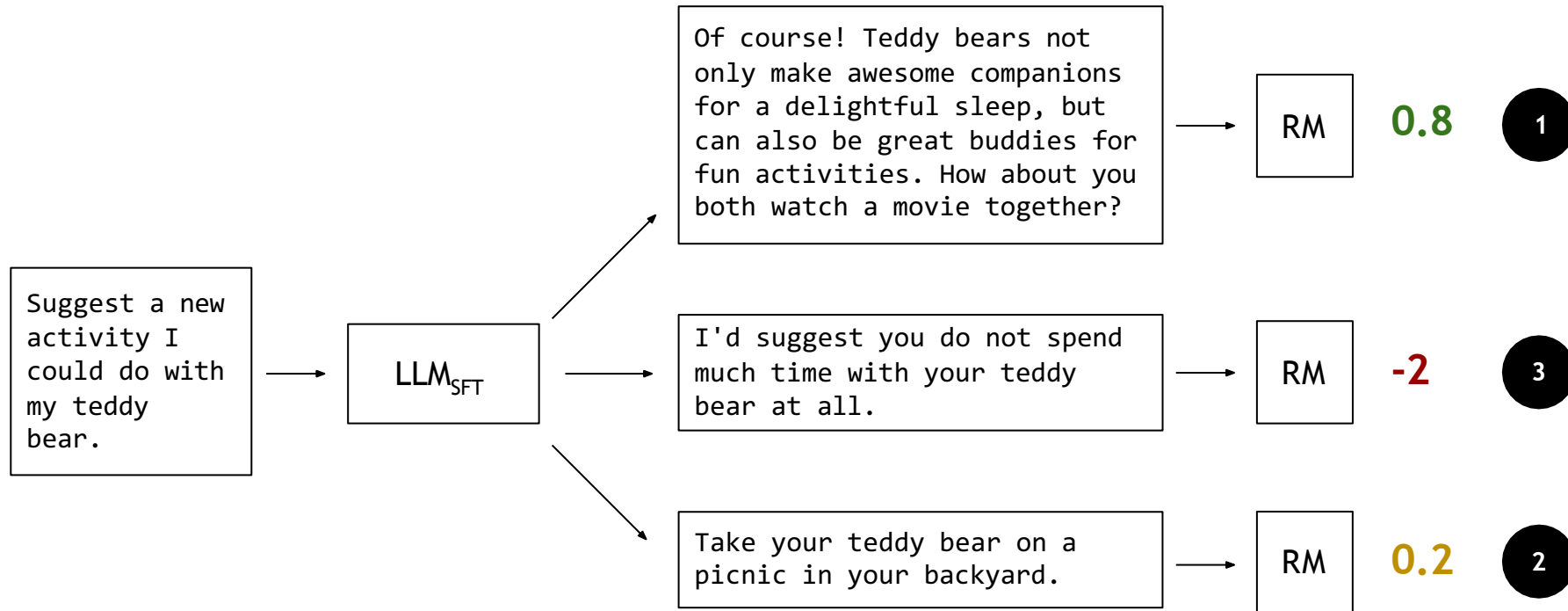
$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\overset{1}{\boxed{\pi_{\theta}(a_t | s_t)}}}{\underset{2}{\boxed{\pi_{\theta_{old}}(a_t | s_t)}}} \overset{3}{\boxed{\hat{A}_t}} - \beta \text{KL}[\overset{2}{\boxed{\pi_{\theta_{old}}(\cdot | s_t)}}, \overset{1}{\boxed{\pi_{\theta}(\cdot | s_t)}}] \right]$$

Some reasonable stuff people thought about

- Train the model with a control token , i.e., SFT on the pairs, prepend [GOOD] to chosen, [BAD] to not chosen.
- Train the model on only preferred output
- Train a reward model, get LM outputs, train on the preferred output
- Train a reward model, get 1024 LM outputs, take the best one (Best-of-N)

Some reasonable stuff people thought about

- Train a reward model, get 1024 LM outputs, take the best one (Best-of-N); too slow...



Direct Preference Optimization (DPO)

- **Try to simplify PPO by..**
 - Getting rid of the reward model
 - Getting rid of any on-policy stuff (rollouts, outer loops etc)
- **Instead**
 - Take gradient steps on log-loss of good stuff
 - Take negative gradient steps on bad stuff (appropriately weighted).

Direct Preference Optimization (DPO)

- DPO loss function

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- No need to train a separate reward model
- Operates directly on preference data
- Like the Bradley-Terry formulation with special kind of reward

$$r_{\theta}(x, y) = \beta \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$$

Let's derive DPO

- Start from PPO Objective

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} [r_{\phi}(x, y)] - \beta \mathbb{D}_{\text{KL}} [\pi_{\theta}(y | x) || \pi_{\text{ref}}(y | x)]$$

Let's derive DPO

- Start from PPO Objective
- Derive optimal policy

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp \left(\frac{1}{\beta} r^*(x, y) \right)$$

Let's derive DPO

- Start from PPO Objective
- Derive optimal policy
- Identify a “reward” term

$$r^*(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

Let's derive DPO

- Start from PPO Objective
- Derive optimal policy
- Identify a “reward” term
- Write Bradley-Terry formulation for this “reward”

$$p^*(y_w \succ y_l \mid x) = \frac{1}{1 + \exp \left(\beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} - \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} \right)}$$

Let's derive DPO

- Start from PPO Objective
- Derive optimal policy
- Identify a “reward” term
- Write Bradley-Terry formulation for this “reward”
- “Infer” DPO loss function

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Comparison

RLHF	DPO
• Multi-stage training • Needs extra models: reward model, value model, base model	• Supervised learning • Base model is the only extra model needed

Performance. No common absolute consensus. Varies from task to task and sensitive to implementation.

Behavior

Can I put my
teddy bear in
the washer?



**Pretrained +
instruction
tuned LLM**



No, it might get
damaged. Try hand
washing instead.

Behavior

Can I put my
teddy bear in
the washer?

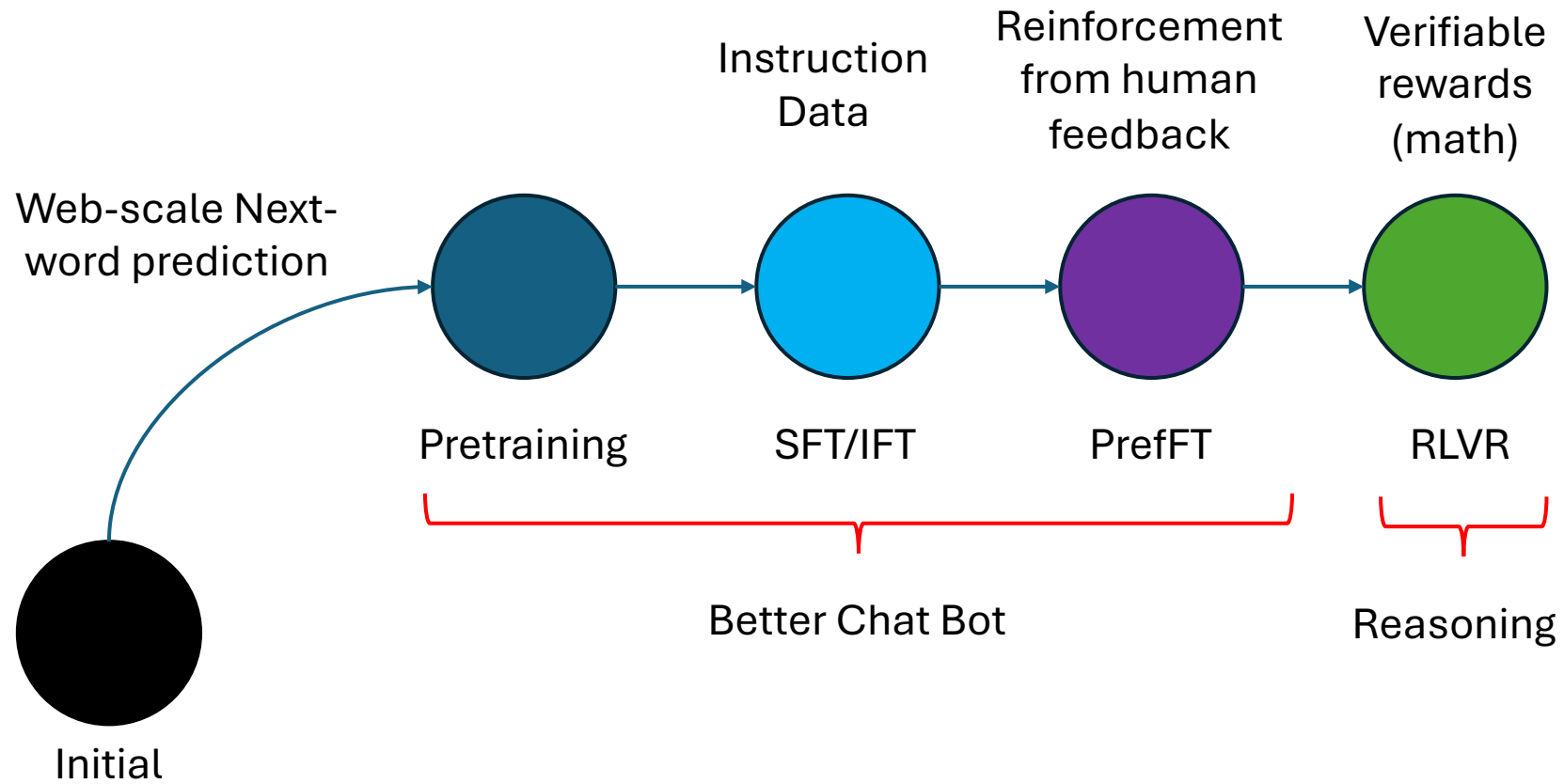


**Pretrained +
instruction
tuned +
preference
tuned LLM**



It's better not to.
Your teddy could get
hurt! A gentle hand
wash is safer.

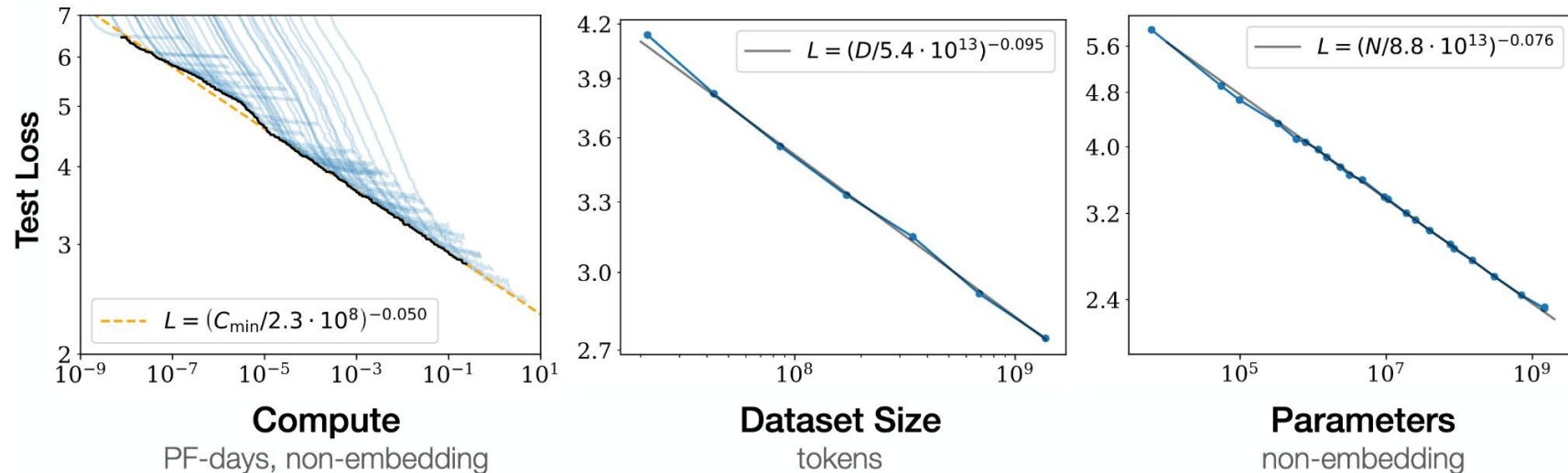
Next week, reasoning!



Appendix

Scaling laws

- The more compute you have the better it is
- The bigger the models the better the model is



Sample efficiency

